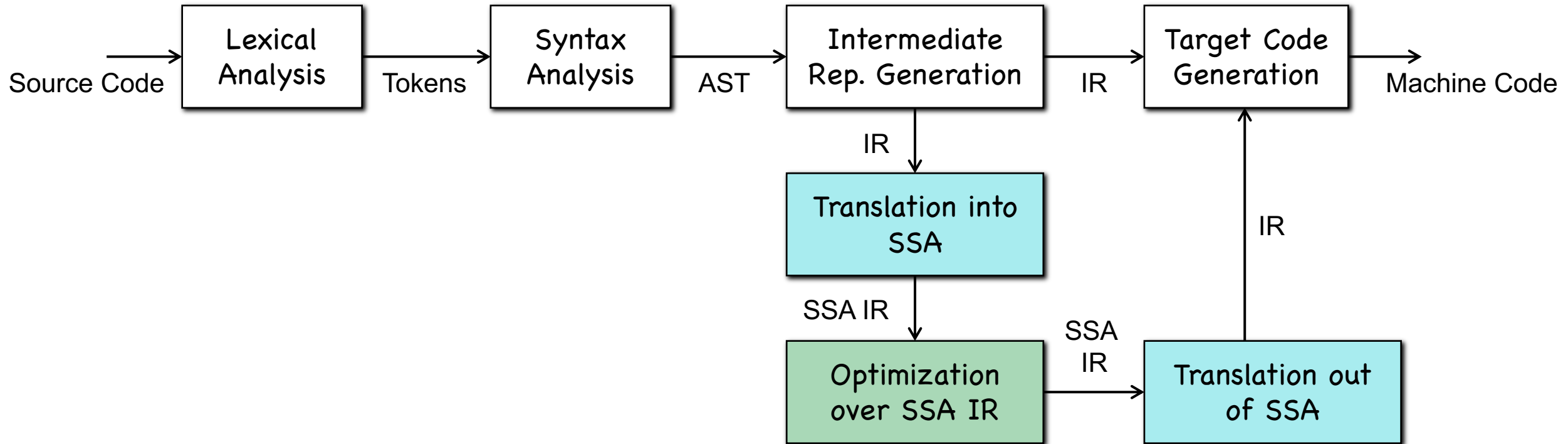


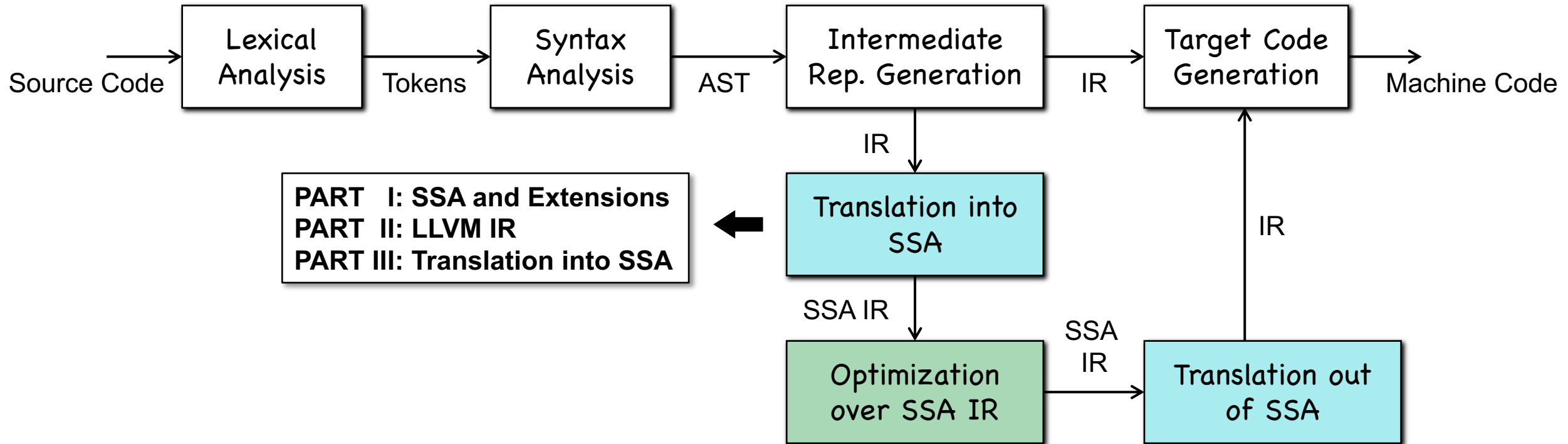
Chapter 6-2

Translation into SSA Form

SSA in Compilers



SSA in Compilers



PART I: SSA and Extensions

Recap: Three-Address Code

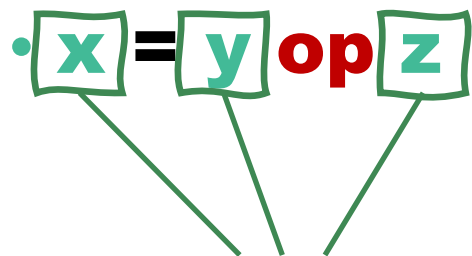
- Provide a standard representation, not as flexible as the source

Recap: Three-Address Code

- Provide a standard representation, not as flexible as the source
- At most one operator on the right side of each instruction
- At most three **addresses/variables** in an instruction
 - $x = y \text{ op } z$

Recap: Three-Address Code

- Provide a standard representation, not as flexible as the source
- At most one operator on the right side of each instruction
- At most three **addresses/variables** in an instruction



Variables or constants;

Variables are from the source code or created by the compiler

Recap: Three-Address Code

- **do** $i = i + 1$; **while** $(a[i + 2] < v)$;

```
L:  t1 = i + 1  
    i = t1  
    t2 = i + 2  
    t3 = a [ t2 ]  
    if t3 < v goto L
```

Symbolic Labels

```
100: t1 = i + 1  
101: i = t1  
102: t2 = i + 2  
103: t3 = a [ t2 ]  
104: if t3 < v goto 100
```

Numeric Labels

Static Single-Assignment

- What features does SSA have?

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

(a) Three-address code.

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains

```

p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
  
```

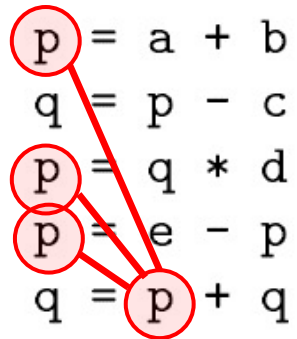
```

p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
  
```

(a) Three-address code. (b) Static single-assignment form.

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains



(a) Three-address code.

```

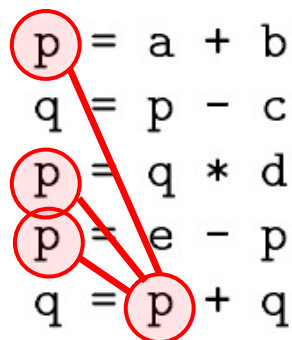
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1

```

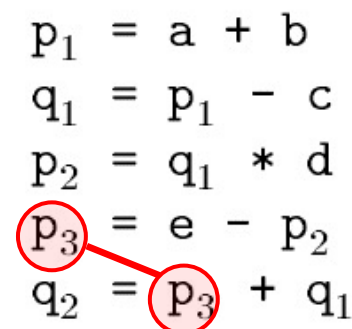
(b) Static single-assignment form.

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains



(a) Three-address code.



(b) Static single-assignment form.

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains

Static Single-Assignment

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using ϕ to merge definitions from multi paths
- $\Rightarrow$ Direct def-use chains

```
if ( flag ) x = -1; else x = 1;
y = x * a;
```



```
if ( flag ) x1 = -1; else x2 = 1;
x3 =  $\phi$  (x1, x2);
y = x3 * a
```

Static Single-Assignment

- $X = 1; \text{ while } (X < N) \{ \text{ if } (P) X = X + 1; \}$

Try!

Static Single-Assignment

- $X = 1; \text{ while } (X < N) \{ \text{ if } (P) X = X + 1; \}$

```
    X = 1
H:   if  $X \geq N$  goto E
      if  $\neg P$  goto H
       $X = X + 1$ 
      goto H
E:   ....
```

Static Single-Assignment

- $X = 1; \text{ while } (X < N) \{ \text{ if } (P) X = X + 1; \}$

```

    X = 1
H:   if  $X \geq N$  goto E
      if  $\neg P$  goto H
       $X = X + 1$ 
      goto H
E:   ....

```



```

     $X_1 = 1$ 
H:    $X_2 = \phi(X_1, X_4)$ 
      if  $X_2 \geq N$  goto E
      if  $\neg P$  goto L
       $X_3 = X_2 + 1$ 
L:    $X_4 = \phi(X_2, X_3)$ 
      goto H
E:   ....

```

Gated SSA

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using γ to merge definitions from multi-paths
- $\Rightarrow$ Direct def-use chains + conditional def-use chains

Gated SSA

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Using γ to merge definitions from multi-paths
- $\Rightarrow$ Direct def-use chains + conditional def-use chains

```
if ( flag ) x = -1; else x = 1;
y = x * a;
```



```
if ( flag ) x1 = -1; else x2 = 1;
x3 =  $\gamma$  (flag, x1, x2);
y = x3 * a
```

Gated SSA

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Use recursive γ to merge definitions from multi-paths
- $\Rightarrow$ Direct def-use chains + conditional def-use chains

Gated SSA

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Use recursive γ to merge definitions from multi-paths
- $\Rightarrow$ Direct def-use chains + conditional def-use chains

if (flag_1) { $x_1 = 1$; }

else if (flag_2) { $x_2 = 2$; }

else { $x_3 = 3$; }

$x_4 = \gamma (\text{flag}_1, x_1, \gamma (\text{flag}_2, x_2, x_3));$

Gated SSA

- **Feature 1:** Every variable has only one definition
- **Feature 2:** Use **recursive** γ to merge definitions from multi-paths
- **Feature 3:** Use μ and η for loop variables

Gated SSA

- **Feature 3:** Using μ and η for loop variables
- $X = 1$; **while** ($X < N$) { **if** (P) $X = X + 1$; }

```

X = 1
H:  if X ≥ N goto E
    if ¬P goto H
    X = X + 1
    goto H
E:  ....

```



SSA

```

X1 = 1
H:  X2 = φ (X1, X4)
    if X2 ≥ N goto E
    if ¬P goto L
    X3 = X2 + 1
L:  X4 = φ (X2, X3)
    goto H
E:  ....

```



GSA

```

X1 = 1
H:  X2 = μ (X1, X4)
    if X2 ≥ N goto E
    if ¬P goto L
    X3 = X2 + 1
L:  X4 = γ (P, X3, X2)
    goto H
E:  X5 = η (X2 ≥ N, X2)

```

Research on SSA

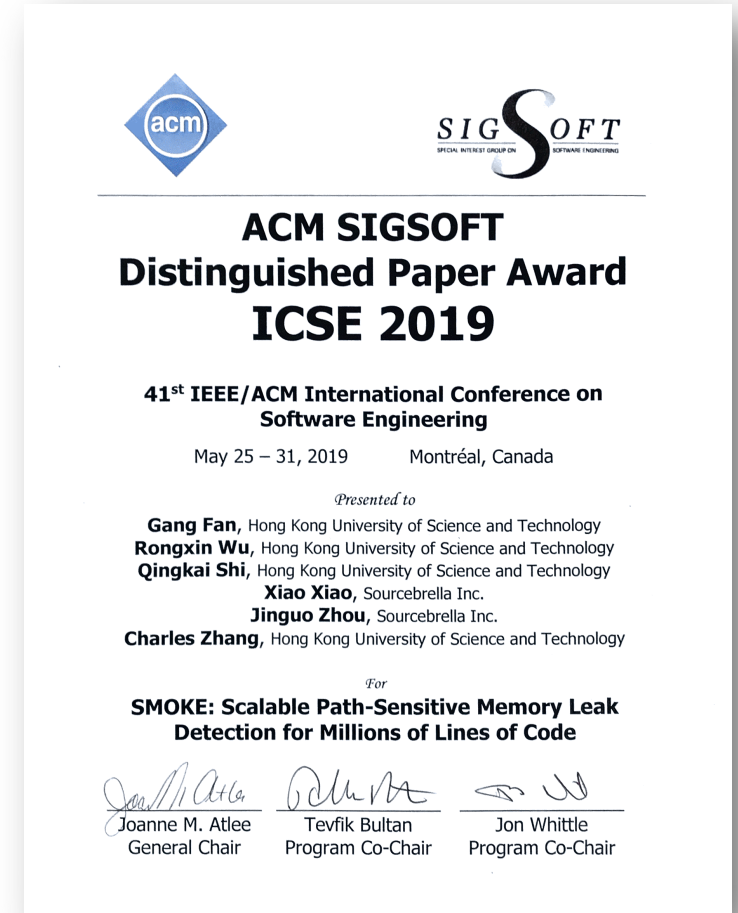
- More powerful extension of SSA
- Program analysis via SSA
- Compiler optimization via SSA
- Theory of SSA

Extensions of SSA

- SSA (1989)
- Gated SSA (1993)
- Hashed SSA (1996)
- SSI: Static Single Information (1999)
- IPSSA (2003)

Extensions of SSA

- SSA (1989)
- Gated SSA (1993)
- Hashed SSA (1996)
- SSI: Static Single Information (1999)
- IPSSA (2003)
- SSU: Static Single Use (2019)
 - ACM SIGSOFT Distinguished Paper Award



SSA Complexity

- Intuitively, code in SSA is much larger than the non-SSA version

SSA Complexity

- Intuitively, code in SSA is much larger than the non-SSA version
- $X = 1$; **while** ($X < N$) { **if** (P) $X = X + 1$; }

```

    X = 1
H:   if X ≥ N goto E
      if ¬P goto H
      X = X + 1
      goto H
E:   ....

```



```

    X1 = 1
H:   X2 = φ (X1, X4)
      if X2 ≥ N goto E
      if ¬P goto L
      X3 = X2 + 1
L:   X4 = φ (X2, X3)
      goto H
E:   ....

```

SSA Complexity

- Space
 - Common three-address code vs. SSA code
 - **Almost Linear!**
- Time
 - From common three-address code to SSA code
 - **Almost Linear!**
- **How? What's the Algorithm?**

SSA Complexity

- Space
 - Common three-address code vs. SSA code
 - **Almost Linear!**
- Time
 - From common three-address code to SSA code
 - **Almost Linear!**
- **How? What's the Algorithm?**



Compilers using SSA

SSA form is a relatively recent development in the compiler community. As such, many older compilers only use SSA form for some part of the compilation or optimization process, but most do not rely on it. Examples of compilers that rely heavily on SSA form include:

- The ETH [Oberon-2](#) compiler was one of the first public projects to incorporate "GSA", a variant of SSA.
- The [LLVM](#) Compiler Infrastructure uses SSA form for all scalar register values (everything except memory) in its primary code representation. SSA form is only eliminated once register allocation occurs, late in the compile process (often at link time).
- The [Open64](#) compiler uses SSA form in its global scalar optimizer, though the code is brought into SSA form before and taken out of SSA form afterwards. Open64 uses extensions to SSA form to represent memory in SSA form as well as scalar values.
- Since the version 4 (released in April 2005) [GCC](#), the [GNU Compiler Collection](#), makes extensive use of SSA. The [frontends](#) generate "GENERIC" code that is then converted into "GIMPLE" code by the "gimplifier". High-level optimizations are then applied on the SSA form of "GIMPLE". The resulting optimized intermediate code is then translated into [RTL](#), on which low-level optimizations are applied. The architecture-specific [backends](#) finally turn RTL into [assembly language](#).
- IBM's open source adaptive [Java virtual machine](#), [Jikes RVM](#), uses extended Array SSA, an extension of SSA that allows analysis of scalars, arrays, and object fields in a unified framework. Extended Array SSA analysis is only enabled at the maximum optimization level, which is applied to the most frequently executed portions of code.
- In 2002, [researchers modified](#) IBM's JikesRVM (named Jalapeño at the time) to run both standard Java [bytecode](#) and a typesafe SSA ([SafeTSA](#)) bytecode class files, and demonstrated significant performance benefits to using the SSA bytecode.
- Oracle's [HotSpot Java Virtual Machine](#) uses an SSA-based intermediate language in its JIT compiler.^[18]
- Microsoft [Visual C++](#) compiler backend available in [Microsoft Visual Studio](#) 2015 Update 3 uses SSA^[19]
- [Mono](#) uses SSA in its JIT compiler called Mini.
- [jackcc](#) is an open-source compiler for the academic instruction set Jackal 3.0. It uses a simple 3-operand code with SSA for its intermediate representation. As an interesting variant, it replaces Φ functions with a so-called SAME instruction, which instructs the register allocator to place the two live ranges into the same physical register.
- Although not a compiler, the [Boomerang](#) decompiler uses SSA form in its internal representation. SSA is used to simplify expression propagation, identifying parameters and returns, preservation analysis, and more.

- **GCC**
- **CLANG**
- **RustC**
- ...



PART II: LLVM IR

LLVM and Clang

<https://llvm.org/>



LLVM and Clang <https://llvm.org/>

- From the beginning of this century, now → version 20.x
- LLVM Compiler Infrastructure → Clang/Clang++/....



LLVM and Clang <https://llvm.org/>

- From the beginning of this century, Jan 23rd → version 18.x
- LLVM Compiler Infrastructure → Clang/Clang++/....
- Core: LLVM IR (Bitcode) and Utilities
- Static Single Assignment (SSA) style IR



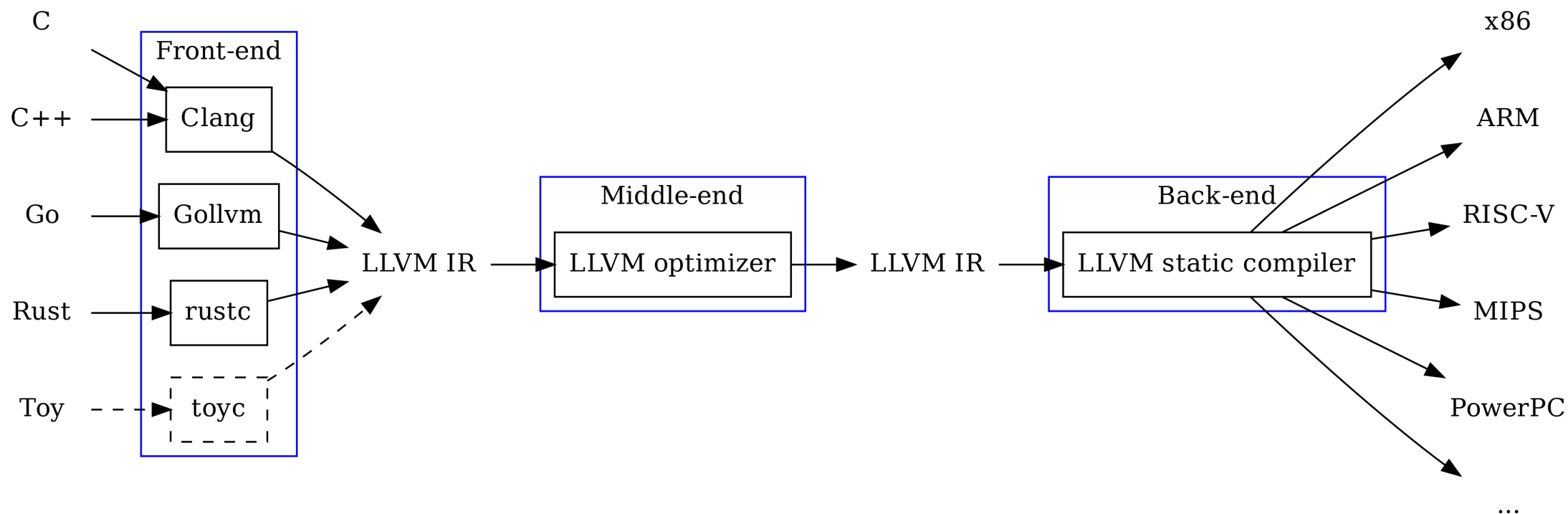
LLVM and Clang <https://llvm.org/>

- From the beginning of this century, Jan 23rd → version 18.x
- LLVM Compiler Infrastructure → Clang/Clang++/....
- Core: LLVM IR (Bitcode) and Utilities
- Static Single Assignment (SSA) style IR
- Analysis, Optimization, Testing, Debugging,



LLVM and Clang <https://llvm.org/>

- LLVM defines a set of APIs to manipulate the LLVM IR



LLVM IR / Bitcode

<https://llvm.org/docs/LangRef.html>

- `clang -c -emit-llvm hello.c [-o hello.bc]/[-S -o hello.ll]`

LLVM IR / Bitcode

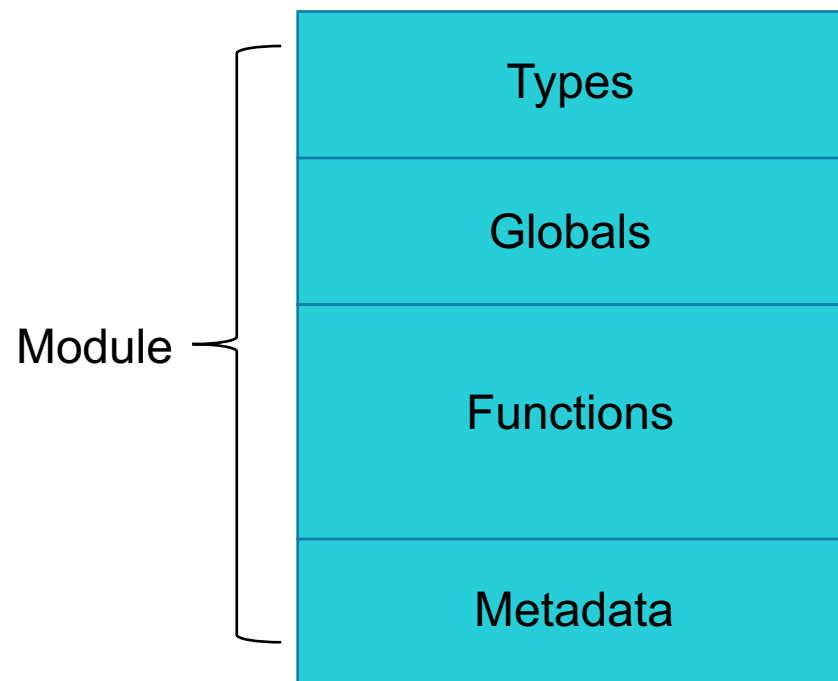
<https://llvm.org/docs/LangRef.html>

- `clang -c -emit-llvm hello.c [-o hello.bc]/[-S -o hello.ll]`
- `llvm-dis hello.bc -o hello.ll` `llvm-as hello.ll -o hello.bc`

LLVM IR / Bitcode

<https://llvm.org/docs/LangRef.html>

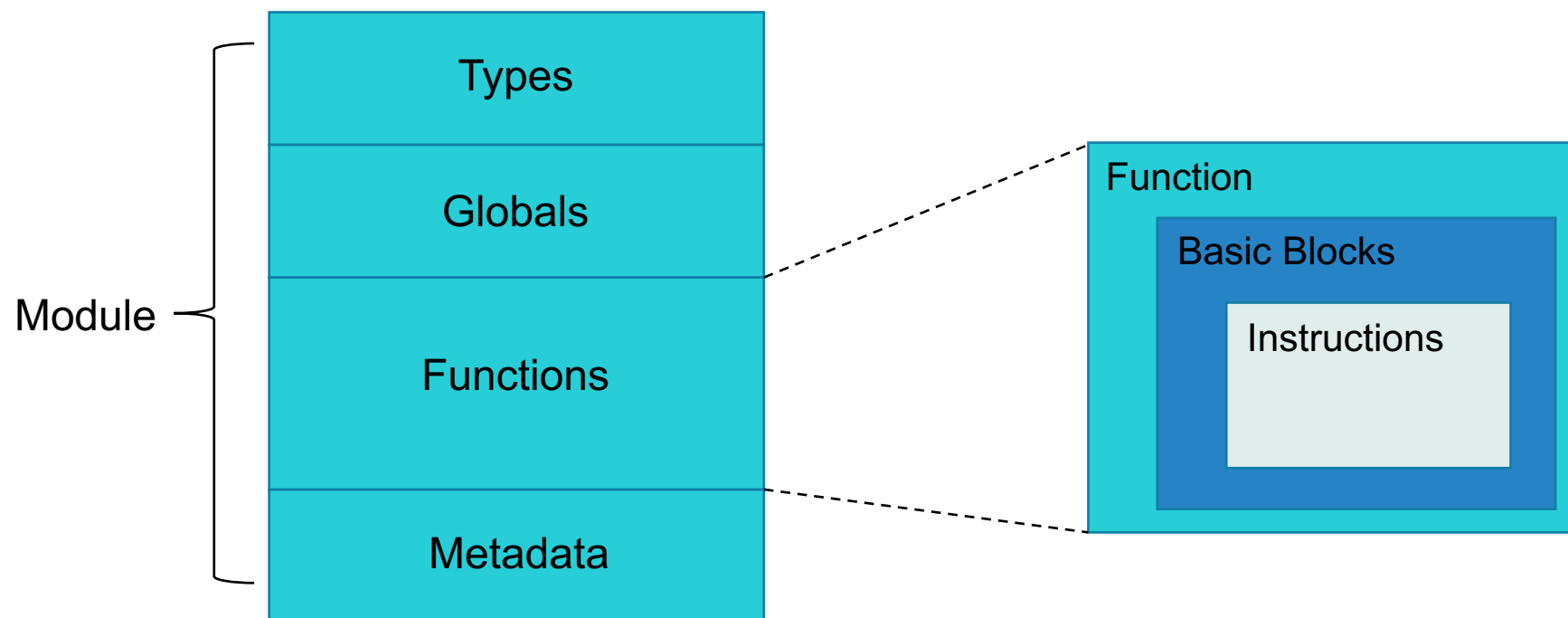
- `clang -c -emit-llvm hello.c [-o hello.bc]/[-S -o hello.ll]`
- `llvm-dis hello.bc -o hello.ll` `llvm-as hello.ll -o hello.bc`



LLVM IR / Bitcode

<https://llvm.org/docs/LangRef.html>

- `clang -c -emit-llvm hello.c [-o hello.bc]/[-S -o hello.ll]`
- `llvm-dis hello.bc -o hello.ll` `llvm-as hello.ll -o hello.bc`



LLVM Function

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {  
    if (val == 0)  
        return 1;  
    return val * factorial(val - 1);  
}
```

LLVM Function

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {  
    if (val == 0)  
        return 1;  
    return val * factorial(val - 1);  
}
```

```
define i32 @factorial(i32 %0) {  
1:  
    %2 = icmp eq i32 %0, 0  
    br i1 %2, label %7, label %3  
  
3:  
    %4 = add i32 %0, -1  
    %5 = call i32 @factorial(i32 %4)  
    %6 = mul i32 %5, %0  
    br label %7  
  
7:  
    %8 = phi i32 [ %6, %3 ], [ 1, %1 ]  
    ret i32 %8  
}
```

LLVM Function

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {  
    if (val == 0)  
        return 1;  
    return val * factorial(val - 1);  
}
```

```
define i32 @factorial(i32 %0) {  
1:  
    %2 = icmp eq i32 %0, 0  
    br i1 %2, label %7, label %3  
  
3:  
    %4 = add i32 %0, -1  
    %5 = call i32 @factorial(i32 %4)  
    %6 = mul i32 %5, %0  
    br label %7  
  
7:  
    %8 = phi i32 [ %6, %3 ], [ 1, %1 ]  
    ret i32 %8  
}
```

LLVM Function

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {  
    if (val == 0)  
        return 1;  
    return val * factorial(val - 1);  
}
```

```
define i32 @factorial(i32 %0) {  
1:
```

```
    %2 = icmp eq i32 %0, 0  
    br i1 %2, label %7, label %3
```

```
3:
```

```
    %4 = add i32 %0, -1  
    %5 = call i32 @factorial(i32 %4)  
    %6 = mul i32 %5, %0  
    br label %7
```

```
7:
```

```
    %8 = phi i32 [ %6, %3 ], [ 1, %1 ]  
    ret i32 %8  
}
```


LLVM Function

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
    return val * factorial(val - 1);
}
```

```
define i32 @factorial(i32 %0) {
1:
    %2 = icmp eq i32 %0, 0
    br i1 %2, label %7, label %3
```

3:

```
    %4 = add i32 %0, -1
    %5 = call i32 @factorial(i32 %4)
    %6 = mul i32 %5, %0
    br label %7
```

7:

```
    %8 = phi i32 [ %6, %3 ], [ 1, %1 ]
    ret i32 %8
}
```

LLVM Function

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
    return val * factorial(val - 1);
}
```

```
define i32 @factorial(i32 %0) {
1:
    %2 = icmp eq i32 %0, 0
    br i1 %2, label %7, label %3

3:
    %4 = add i32 %0, -1
    %5 = call i32 @factorial(i32 %4)
    %6 = mul i32 %5, %0
    br label %7

7:
    %8 = phi i32 [ %6, %3 ], [ 1, %1 ]
    ret i32 %8
}
```

LLVM Metadata

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
    return val * factorial(val - 1);
}
```

```
define i32 @factorial(i32 %0) !dbg !7 {
1:
    %2 = icmp eq i32 %0, 0, !dbg !14
    br i1 %2, label %7, label %3, !dbg !16

3:
    %4 = add i32 %0, -1, !dbg !17
    %5 = call i32 @factorial(i32 %4), !dbg !18
    %6 = mul i32 %5, %0, !dbg !19
    br label %7, !dbg !20

7:
    %8 = phi i32 [ %6, %3 ], [ 1, %1 ], !dbg !13
    ret i32 %8, !dbg !21
}
```

```
clang -g -c -emit-llvm hello.c -S -o hello.ll
```

LLVM Metadata

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
```

```
define i32 @factorial(i32 %0) !dbg !7 {
1:
    %2 = icmp eq i32 %0, 0, !dbg !14
```

```
!0 = distinct !DICompileUnit(language: DW_LANG_C99, file: !1, producer: "Ubuntu clang v
d: FullDebug, enums: !2, splitDebugInlining: false, nameTableKind: None)
!1 = !DIFile(filename: "hello.c", directory: "/home/qingkaishi")
!2 = !{}
!3 = !{i32 7, !"Dwarf Version", i32 4}
!4 = !{i32 2, !"Debug Info Version", i32 3}
!5 = !{i32 1, !"wchar_size", i32 4}
!6 = !{"Ubuntu clang version 12.0.1-19ubuntu3"}
!7 = distinct !DISubprogram(name: "factorial", scope: !1, file: !1, line: 3, type: !8,
ISPFlagDefinition | DISPFlagOptimized, unit: !0, retainedNodes: !11)
!8 = !DISubroutineType(types: !9)
!9 = !{!10, !10}
!10 = !DIBasicType(name: "int", size: 32, encoding: DW_ATE_signed)
!11 = !{!12}
!12 = !DILocalVariable(name: "val", arg: 1, scope: !7, file: !1, line: 3, type: !10)
!13 = !DILocation(line: 0, scope: !7)
!14 = !DILocation(line: 4, column: 13, scope: !15)
!15 = distinct !DILexicalBlock(scope: !7, file: !1, line: 4, column: 9)
!16 = !DILocation(line: 4, column: 9, scope: !7)
```

!dbg !16

!dbg !18

!dbg !13

—o hello.ll

LLVM Metadata

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
```

```
define i32 @factorial(i32 %0) !dbg !7 {
1:
    %2 = icmp eq i32 %0, 0, !dbg !14
```

```
!0 = distinct !DICompileUnit(language: DW_LANG_C99, file: !1, producer: "Ubuntu clang v
d: FullDebug, enums: !2, splitDebugInlining: false, nameTableKind: None)
!1 = !DIFile(filename: "hello.c", directory: "/home/qingkaishi")
!2 = !{}
!3 = !{i32 7, !"Dwarf Version", i32 4}
!4 = !{i32 2, !"Debug Info Version", i32 3}
!5 = !{i32 1, !"wchar_size", i32 4}
!6 = !{"Ubuntu clang version 12.0.1-19ubuntu3"}
!7 = distinct !DISubprogram(name: "factorial", scope: !1, file: !1, line: 3, type: !8,
ISPFflagDefinition | DISPFflagOptimized, unit: !0, retainedNodes: !11)
!8 = !DISubroutineType(types: !9)
!9 = !{!10, !10}
!10 = !DIBasicType(name: "int", size: 32, encoding: DW_ATE_signed)
!11 = !{!12}
!12 = !DILocalVariable(name: "val", arg: 1, scope: !7, file: !1, line: 3, type: !10)
!13 = !DILocation(line: 0, scope: !7)
!14 = !DILocation(line: 4, column: 13, scope: !15)
!15 = distinct !DILexicalBlock(scope: !7, file: !1, line: 4, column: 9)
!16 = !DILocation(line: 4, column: 9, scope: !7)
```

!dbg !16

!dbg !18

!dbg !13

!dbg !13

LLVM Metadata

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
```

```
define i32 @factorial(i32 %0) !dbg !7 {
1:
    %2 = icmp eq i32 %0, 0, !dbg !14
```

```
!0 = distinct !DICompileUnit(language: DW_LANG_C99, file: !1, producer: "Ubuntu clang v
d: FullDebug, enums: !2, splitDebugInlining: false, nameTableKind: None)
!1 = !DIFile(filename: "hello.c", directory: "/home/qingkaishi")
!2 = !{}
!3 = !{i32 7, !"Dwarf Version", i32 4}
!4 = !{i32 2, !"Debug Info Version", i32 3}
!5 = !{i32 1, !"wchar_size", i32 4}
!6 = !{"Ubuntu clang version 12.0.1-19ubuntu3"}
!7 = distinct !DISubprogram(name: "factorial", scope: !1, file: !1, line: 3, type: !8,
ISPFlagDefinition | DISPFlagOptimized, unit: !0, retainedNodes: !11)
!8 = !DISubroutineType(types: !9)
!9 = !{!10, !10}
!10 = !DIBasicType(name: "int", size: 32, encoding: DW_ATE_signed)
!11 = !{!12}
!12 = !DILocalVariable(name: "val", arg: 1, scope: !7, file: !1, line: 3, type: !10)
!13 = !DILocation(line: 0, scope: !7)
!14 = !DILocation(line: 4, column: 13, scope: !15)
!15 = distinct !DILexicalBlock(scope: !7, file: !1, line: 4, column: 9)
!16 = !DILocation(line: 4, column: 9, scope: !7)
```

!dbg !16

!dbg !18

!dbg !13

—o hello.ll

LLVM Metadata

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
```

```
define i32 @factorial(i32 %0) !dbg !7 {
1:
    %2 = icmp eq i32 %0, 0, !dbg !14
```

```
!0 = distinct !DICompileUnit(language: DW_LANG_C99, file: !1, producer: "Ubuntu clang v
d: FullDebug, enums: !2, splitDebugInlining: false, nameTableKind: None)
!1 = !DIFile(filename: "hello.c", directory: "/home/qingkaishi")
!2 = !{}
!3 = !{i32 7, !"Dwarf Version", i32 4}
!4 = !{i32 2, !"Debug Info Version", i32 3}
!5 = !{i32 1, !"wchar_size", i32 4}
!6 = !{"Ubuntu clang version 12.0.1-19ubuntu3"}
!7 = distinct !DISubprogram(name: "factorial", scope: !1, file: !1, line: 3, type: !8,
ISPFlagDefinition | DISPFlagOptimized, unit: !0, retainedNodes: !11)
!8 = !DISubroutineType(types: !9)
!9 = !{!10, !10}
!10 = !DIBasicType(name: "int", size: 32, encoding: DW_ATE_signed)
!11 = !{!12}
!12 = !DILocalVariable(name: "val", arg: 1, scope: !7, file: !1, line: 3, type: !10)
!13 = !DILocation(line: 0, scope: !7)
!14 = !DILocation(line: 4, column: 13, scope: !15)
!15 = distinct !DILexicalBlock(scope: !7, file: !1, line: 4, column: 9)
!16 = !DILocation(line: 4, column: 9, scope: !7)
```

!dbg !16

!dbg !18

!dbg !13

—o hello.ll

LLVM Metadata

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
```

```
define i32 @factorial(i32 %0) !dbg !7 {
1:
    %2 = icmp eq i32 %0, 0, !dbg !14
```

```
!0 = distinct !DICompileUnit(language: DW_LANG_C99, file: !1, producer: "Ubuntu clang v
d: FullDebug, enums: !2, splitDebugInlining: false, nameTableKind: None)
!1 = !DIFile(filename: "hello.c", directory: "/home/qingkaishi")
!2 = !{}
!3 = !{i32 7, !"Dwarf Version", i32 4}
!4 = !{i32 2, !"Debug Info Version", i32 3}
!5 = !{i32 1, !"wchar_size", i32 4}
!6 = !{"Ubuntu clang version 12.0.1-19ubuntu3"}
!7 = distinct !DISubprogram(name: "factorial", scope: !1, file: !1, line: 3, type: !8,
ISPFlagDefinition | DISPFlagOptimized, unit: !0, retainedNodes: !11)
!8 = !DISubroutineType(types: !9)
!9 = !{!10, !10}
!10 = !DIBasicType(name: "int", size: 32, encoding: DW_ATE_signed)
!11 = !{!12}
!12 = !DILocalVariable(name: "val", arg: 1, scope: !7, file: !1, line: 3, type: !10)
!13 = !DILocation(line: 0, scope: !7)
!14 = !DILocation(line: 4, column: 13, scope: !15)
!15 = distinct !DILexicalBlock(scope: !7, file: !1, line: 4, column: 9)
!16 = !DILocation(line: 4, column: 9, scope: !7)
```

!dbg !16

!dbg !18

!dbg !13

—o hello.ll

LLVM Metadata

<https://llvm.org/docs/LangRef.html>

```
int factorial(int val) {
    if (val == 0)
        return 1;
}
```

```
define i32 @factorial(i32 %0) !dbg !7 {
1:
    %2 = icmp eq i32 %0, 0, !dbg !14
```

```
!0 = distinct !DICompileUnit(language: DW_LANG_C99, file: !1, producer: "Ubuntu clang v
d: FullDebug, enums: !2, splitDebugInlining: false, nameTableKind: None)
!1 = !DIFile(filename: "hello.c", directory: "/home/qingkaishi")
!2 = !{}
!3 = !{i32 7, !"Dwarf Version", i32 4}
!4 = !{i32 2, !"Debug Info Version", i32 3}
!5 = !{i32 1, !"wchar_size", i32 4}
!6 = !{"Ubuntu clang version 12.0.1-19ubuntu3"}
!7 = distinct !DISubprogram(name: "factorial", scope: !1, file: !1, line: 3, type: !8,
ISPFlagDefinition | DISPFlagOptimized, unit: !0, retainedNodes: !11)
!8 = !DISubroutineType(types: !9)
!9 = !{!10, !10}
!10 = !DIBasicType(name: "int", size: 32, encoding: DW_ATE_signed)
!11 = !{!12}
!12 = !DILocalVariable(name: "val", arg: 1, scope: !7, file: !1, line: 3, type: !10)
!13 = !DILocation(line: 0, scope: !7)
!14 = !DILocation(line: 4, column: 13, scope: !15)
!15 = distinct !DILexicalBlock(scope: !7, file: !1, line: 4, column: 9)
!16 = !DILocation(line: 4, column: 9, scope: !7)
```

!dbg !16

!dbg !18

!dbg !13

!o hello.ll

LLVM Types

<https://llvm.org/docs/LangRef.html>

- Void Type
 - void
- Primitive Types
 - i32, i8, i1, ...
 - float, double, ...

LLVM Types

<https://llvm.org/docs/LangRef.html>

- Void Type
 - void
- Primitive Types
 - i32, i8, i1, ...
 - float, double, ...
- Function Type
 - return-type (parameter types)
 - i32 (i32)

LLVM Types

<https://llvm.org/docs/LangRef.html>

- Void Type
 - void
- Primitive Types
 - i32, i8, i1, ...
 - float, double, ...
- Function Type
 - return-type (parameter types)
 - i32 (i32)
- Pointer Type
 - i32*, [40 x i32]*
 - ptr
- Array Type
 - [40 x i32]
 - [30 x i32*]

LLVM Types

<https://llvm.org/docs/LangRef.html>

- Void Type
 - void
- Primitive Types
 - i32, i8, i1, ...
 - float, double, ...
- Function Type
 - return-type (parameter types)
 - i32 (i32)
- Pointer Type
 - i32*, [40 x i32]*
 - ptr
- Array Type
 - [40 x i32]
 - [30 x i32*]
- Structure Type
 - { i32, [3 x i32], i8 }
 - ...

LLVM GEP

<https://llvm.org/docs/LangRef.html>

```
struct RT {  
    char A;  
    int B[10][20];  
    char C;  
};
```

```
struct ST {  
    int X;  
    double Y;  
    struct RT Z;  
};
```

```
int foo(struct ST *s) {  
    return s[1].Z.B[5][13];  
}
```

LLVM GEP

<https://llvm.org/docs/LangRef.html>

```
struct RT {
    char A;
    int B[10][20];
    char C;
};
```

```
%struct.RT = type { i8, [10 x [20 x i32]], i8 }
%struct.ST = type { i32, double, %struct.RT }
```

```
struct ST {
    int X;
    double Y;
    struct RT Z;
};
```

```
int foo(struct ST *s) {
    return s[1].Z.B[5][13];
}
```

LLVM GEP

<https://llvm.org/docs/LangRef.html>



```

struct RT {
  char A;
  int B[10][20];
  char C;
};

struct ST {
  int X;
  double Y;
  struct RT Z;
};

int foo(struct ST *s) {
  return s[1].Z.B[5][13];
}

%struct.RT = type { i8 [10 x [20 x i32]], i8 }
%struct.ST = type { i32, double, %struct.RT }
  
```


LLVM GEP

<https://llvm.org/docs/LangRef.html>



```

struct RT {
  char A;
  int B[10][20];
  char C;
};

struct ST {
  int X;
  double Y;
  struct RT Z;
};

int foo(struct ST *s) {
  return s[1].Z.B[5][13];
}

%struct.RT = type { i8, [10 x [20 x i32]], i8 }
%struct.ST = type { i32, double, %struct.RT }
  
```

LLVM GEP

<https://llvm.org/docs/LangRef.html>

```

struct RT {
    char A;
    int B[10][20];
    char C;
};

struct ST {
    int X;
    double Y;
    struct RT Z;
};

int foo(struct ST *s) {
    return s[1].Z.B[5][13];
}

```

Diagram illustrating the LLVM GEP (GetElementPtr) operation. A blue line connects the `RT` struct definition to the `%struct.RT` type definition. The `RT` struct is defined as:

```

struct RT {
    char A;
    int B[10][20];
    char C;
};

```

The `%struct.RT` type is defined as:

```

%struct.RT = type { i8, [10 x [20 x i32]], i8 }
%struct.ST = type { i32, double, %struct.RT }

```

The `RT` struct is used in the `ST` struct definition and the `foo` function. The `foo` function returns the address of the element at `s[1].Z.B[5][13]`.

LLVM GEP

<https://llvm.org/docs/LangRef.html>

```

struct RT {
    char A;
    int B[10][20];
    char C;
};

struct ST {
    int X;
    double Y;
    struct RT Z;
};

int foo(struct ST *s) {
    return s[1].Z.B[5][13];
}

%struct.RT = type { i8, [10 x [20 x i32]], i8 }
%struct.ST = type { i32, double, %struct.RT }

define i32 @foo(ptr %s) {

```

LLVM GEP

<https://llvm.org/docs/LangRef.html>

```
struct RT {
    char A;
    int B[10][20];
    char C;
};
```

```
struct ST {
    int X;
    double Y;
    struct RT Z;
};
```

```
int foo(struct ST *s) {
    return s[1].Z.B[5][13];
}
```

```
%struct.RT = type { i8, [10 x [20 x i32]], i8 }
%struct.ST = type { i32, double, %struct.RT }
```

```
define i32 @foo(ptr %s) {
    %0 = getelementptr %struct.ST, ptr %s, i64 1, i32 2, i32 1, i64 5, i64 13
    %1 = load i32, ptr %0
    ret i32 %1
}
```

LLVM Intrinsics

- Special functions, working as extension of instructions
- These functions have well known names and semantics

LLVM Intrinsics

- Special functions, working as extension of instructions
- These functions have well known names and semantics

Standard C/C++ Library Intrinsics

- 'llvm.abs.*' Intrinsic
- 'llvm.smax.*' Intrinsic
- 'llvm.smin.*' Intrinsic
- 'llvm.umax.*' Intrinsic
- 'llvm.umin.*' Intrinsic

LLVM Intrinsics

- Special functions, working as extension of instructions
- These functions have well known names and semantics

Standard C/C++ Library Intrinsics

- 'llvm.abs.*' Intrinsic
- 'llvm.smax.*' Intrinsic
- 'llvm.smin.*' Intrinsic
- 'llvm.umax.*' Intrinsic
- 'llvm.umin.*' Intrinsic

Bit Manipulation Intrinsics

- 'llvm.bitreverse.*' Intrinsics
- 'llvm.bswap.*' Intrinsics
- 'llvm.ctpop.*' Intrinsic
- 'llvm.ctlz.*' Intrinsic
- 'llvm.cttz.*' Intrinsic
- 'llvm.fshl.*' Intrinsic
- 'llvm.fshr.*' Intrinsic

LLVM Intrinsics

- Special functions, working as extension of instructions
- These functions have well known names and semantics

```
void foo(void* src, void *dst, long size) {
    memcpy(dst, src, size);
}
```

```
; Function Attrs: nofree nounwind uwtable willreturn
define dso_local void @foo(i8* nocapture readonly %0, i8* nocapture %1, i32 %2) local_unnamed_addr #0 {
    %4 = sext i32 %2 to i64
    call void @llvm.memcpy.p0i8.p0i8.i64(i8* align 1 %1, i8* align 1 %0, i64 %4, i1 false)
    ret void
}
```


LLVM Global Variables <https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

LLVM Global Variables

<https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

```
extern int x;  
int y = 1;  
  
void foo() {  
    printf("x = %d; y = %d\n", x, y);  
}
```

LLVM Global Variables

<https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

```
extern int x;  
int y = 1;  
  
void foo() {  
    printf("x = %d; y = %d\n", x, y);  
}
```

How many global variables in the LLVM IR?

LLVM Global Variables

<https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

```
extern int x;  
int y = 1;  
  
void foo() {  
    printf("x = %d; y = %d\n", x, y);  
}
```

How many global variables in the LLVM IR?

- Explicit global variables
- Constant string/structs/...

LLVM Global Variables <https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

```
@x = external global i32
@y = global i32 1
@.str = private unnamed_addr constant [16 x i8] c"x = %d; y = %d\0A\00"
```

```
define void @foo() {
  %1 = load i32, i32* @x
  %2 = load i32, i32* @y
  %3 = call i32 @printf(@.str, i32 %1, i32 %2)
  ret void
}
```

```
extern int x;
int y = 1;

void foo() {
    printf("x = %d; y = %d\n", x, y);
}
```

LLVM Global Variables <https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

```
@x = external global i32
```

```
@y = global i32 1
```

```
@.str = private unnamed_addr constant [16 x i8] c"x = %d; y = %d\0A\00"
```

```
define void @foo() {  
  %1 = load i32, i32* @x  
  %2 = load i32, i32* @y  
  %3 = call i32 @printf(@.str, i32 %1, i32 %2)  
  ret void  
}
```

```
extern int x;  
int y = 1;  
  
void foo() {  
    printf("x = %d; y = %d\n", x, y);  
}
```

LLVM Global Variables <https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

@x = external global i32

@y = global i32 1

@.str = private unnamed_addr constant [16 x i8] c"x = %d; y = %d\0A\00"

```
define void @foo() {  
  %1 = load i32, i32* @x  
  %2 = load i32, i32* @y  
  %3 = call i32 @printf(@.str, i32 %1, i32 %2)  
  ret void  
}
```

```
extern int x;  
int y = 1;  
  
void foo() {  
    printf("x = %d; y = %d\n", x, y);  
}
```

LLVM Global Variables <https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

@x = external global i32

@y = global i32 1

@.str = private unnamed_addr constant [16 x i8] c"x = %d; y = %d\0A\00"

```
define void @foo() {  
  %1 = load i32, i32* @x  
  %2 = load i32, i32* @y  
  %3 = call i32 @printf(@.str, i32 %1, i32 %2)  
  ret void  
}
```

```
extern int x;  
int y = 1;  
  
void foo() {  
    printf("x = %d; y = %d\n", x, y);  
}
```


LLVM Global Variables <https://llvm.org/docs/LangRef.html>

- All global variables (@...) in LLVM IR are pointers!

@x = external global i32

@y = global i32 1

@.str = private unnamed_addr constant [16 x i8] c"x = %d; y = %d\0A\00"

define void @foo() {

%1 = load i32, i32* @x

%2 = load i32, i32* @y

%3 = call i32 (i8*, ...) @printf(@.str, i32 %1, i32 %2)

ret void

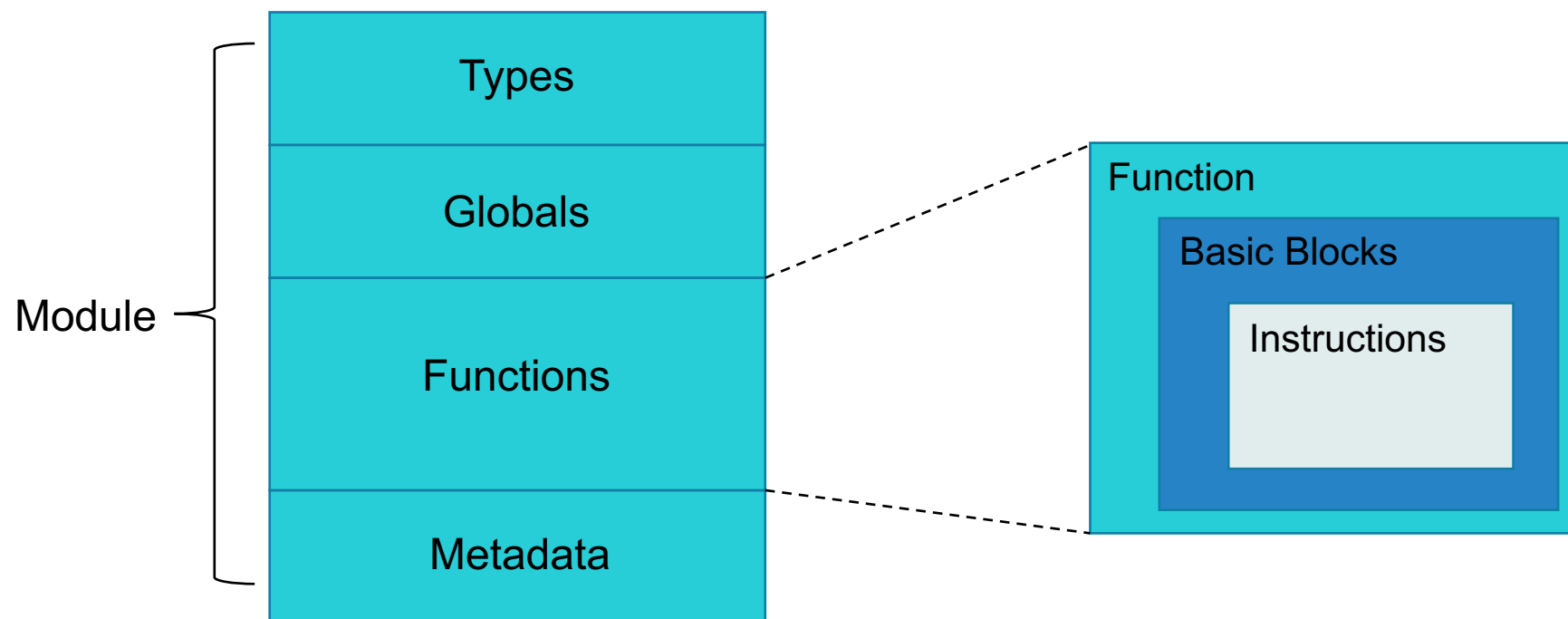
}

```
extern int x;
int y = 1;

void foo() {
    printf("x = %d; y = %d\n", x, y);
}
```

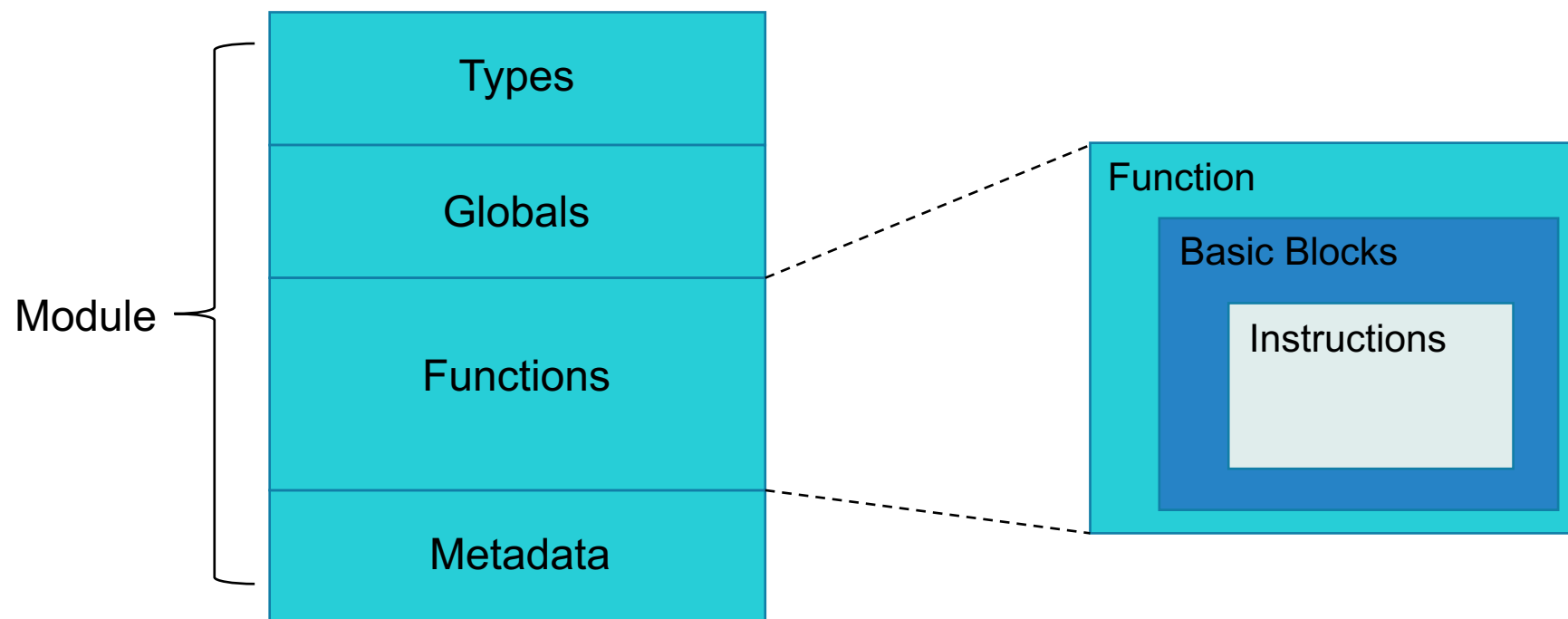
Write a Pass

- `clang -c -emit-llvm hello.c -o hello.bc`
- `llvm-dis hello.bc -o hello.ll` `llvm-as hello.ll -o hello.bc`



Write a Pass

- Legacy Pass <https://llvm.org/docs/WritingAnLLVMPass.html>
- New Pass <https://llvm.org/docs/WritingAnLLVMNewPMPass.html>



Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- ModulePass and FunctionPass

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- ModulePass and FunctionPass

- `// PassA.h`
- `class` PassA : `public` ModulePass {
- `public`:
- `static char` ID;
-
-
- }

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- ModulePass and FunctionPass

- `// PassA.h`
- `class` PassA : `public` ModulePass {
- `public`:
- `static char` ID;
- PassA() : ModulePass(ID) {}
-
- }

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- ModulePass and FunctionPass
- `// PassA.h`
- `class` PassA : `public` ModulePass {
- `public`:
- `static char` ID;
- `PassA() : ModulePass(ID) {}`
- `bool` runOnModule(Module& M) `override`;
- }

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.cpp`
- `#include "PassA.h"`
- `char PassA::ID = 0;`
- `static RegisterPass<PassA> X("pass-a", "...");`

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.cpp`
- `#include "PassA.h"`
- `char PassA::ID = 0;`
- `static RegisterPass<PassA> X("pass-a", "...");`
- `bool PassA::runOnModule(Module &M) {`
 -
 -
 - `}`

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.cpp`
- `#include "PassA.h"`
- `char PassA::ID = 0;`
- `static RegisterPass<PassA> X("pass-a", "...");`
- `bool PassA::runOnModule(Module &M) {`
 - `for (auto &F : M) outs() << F.getName() << "\n";`
 -
 - `}`

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

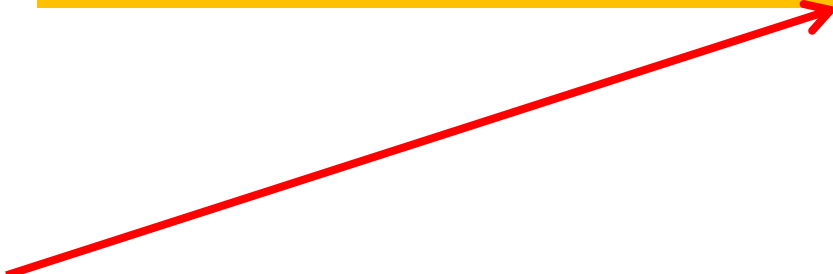
- `// PassA.cpp`
- `#include "PassA.h"`
- `char PassA::ID = 0;`
- `static RegisterPass<PassA> X("pass-a", "...");`
- `bool PassA::runOnModule(Module &M) {`
 - `for (auto &F : M) outs() << F.getName() << "\n";`
 - `return false;`
 - `}`

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.cpp`
- `#include "PassA.h"`
- `char PassA::ID = 0;`
- `static RegisterPass<PassA> X("pass-a", "...");`
- `bool PassA::runOnModule(Module &M) {`
 - `for (auto &F : M) outs() << F.getName() << "\n";`
 - `return false;`
 - `}`

`opt -load pass-a.so --pass-a hello.bc`



Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- A Pass may depend on the results of other LLVM Passes

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- A Pass may depend on the results of other LLVM Passes

```
char get(char *buffer, int size, int index) {  
    return buffer[index];  
}
```

```
char get(char *buffer, int size, int index) {  
    assert (index < size); // to avoid buffer overrun  
    return buffer[index];  
}
```



Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- A Pass may depend on the results of other LLVM Passes

```
char get(char *buffer, int size, int index) {  
    return buffer[index];  
}
```

```
char get(char *buffer, int size, int index) {  
    assert (index < size); // to avoid buffer overrun  
    return buffer[index];  
}
```



Analysis Pass: Find possible buffer overruns



Transform Pass: Insert assertion to avoid overrun

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.h`
- `class` PassA : `public` ModulePass {
- `public`:
- `static char` ID;
- `PassA() : ModulePass(ID) {}`
- `bool` runOnModule(Module& M) `override`;
- `void` getAnalysisUsage(AnalysisUsage &AU) `override`;
- `}`

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.h`
- `class` PassA : `public` ModulePass {
- `public`:
- `static char` ID;
- `PassA()` : ModulePass(ID) {}
- `bool` runOnModule(Module& M) `override`;
- `void` **`getAnalysisUsage(AnalysisUsage &AU)` `override`;**
- }

LLVM's pass manager pre-computes an execution order according to the dependency declared in this function!

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.cpp`
- `.....`
- `void PassA::getAnalysisUsage(AnalysisUsage& AU) {`
- `AU.setPreservesAll();`
- `AU.addRequired<PassB>();`
- `}`

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.cpp`
- `.....`
- `void PassA::getAnalysisUsage(AnalysisUsage& AU) {`
- `AU.setPreservesAll();`
- `AU.addRequired<PassB>();`
- `}`

```
1  class PassB : public ModulePass {  
2  private:  
3      int NumFuncs;  
4  
5  public:  
6      int getNumFunctions() { return NumFuncs; }  
7  
8      bool runOnModule(Module &M) {  
9          NumFuncs = M.size(); return false;  
10     }  
11 }
```

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.cpp`
- `.....`
- `void PassA::getAnalysisUsage(AnalysisUsage& AU) {`
- `AU.setPreservesAll();`
- `AU.addRequired<PassB>();`
- `}`
- `bool PassA::runOnModule(Module &M) {`
- `PassB *PB = &getAnalysis<PassB>();`
- `outs() << PB->getNumFunctions() << "\n";`
- `return false;`
- `}`

```

1  class PassB : public ModulePass {
2  private:
3      int NumFuncs;
4
5  public:
6      int getNumFunctions() { return NumFuncs; }
7
8      bool runOnModule(Module &M) {
9          NumFuncs = M.size(); return false;
10     }
11 }
```

Legacy Pass

<https://llvm.org/docs/WritingAnLLVMPass.html>

- `// PassA.cpp`
- `.....`
- `void PassA::getAnalysisUsage(AnalysisUsage& AU) {`
- `AU.setPreservesAll(); // or AU.addPreserved<PassB>(); for specific passes`
- `AU.addRequired<PassB>();`
- `}`

- `bool PassA::runOnModule(Module &M) {`
- `PassB *PB = &getAnalysis<PassB>();`
- `outs() << PB->getNumFunctions() << "\n";`
- `return false;`
- `}`

New Pass

<https://llvm.org/docs/WritingAnLLVMNewPMPass.html>

New Pass

<https://llvm.org/docs/WritingAnLLVMNewPMPass.html>

```
#ifndef LLVM_TRANSFORMS_HELLOWNEW_HELLOWORLD_H
#define LLVM_TRANSFORMS_HELLOWNEW_HELLOWORLD_H

#include "llvm/IR/PassManager.h"

namespace llvm {

class HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
public:
    PreservedAnalyses run(Function &F, FunctionAnalysisManager &AM);
};

} // namespace llvm

#endif // LLVM_TRANSFORMS_HELLOWNEW_HELLOWORLD_H
```

New Pass

<https://llvm.org/docs/WritingAnLLVMNewPMPass.html>

```
#ifndef LLVM_TRANSFORMS_HELLOWNEW_HELLOWORLD_H
#define LLVM_TRANSFORMS_HELLOWNEW_HELLOWORLD_H

#include "llvm/IR/PassManager.h"

namespace llvm {

class HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
public:
    PreservedAnalyses run(Function &F, FunctionAnalysisManager &AM);
};

} // namespace llvm

#endif // LLVM_TRANSFORMS_HELLOWNEW_HELLOWORLD_H
```


New Pass

<https://llvm.org/docs/WritingAnLLVMNewPMPass.html>

```
PreservedAnalyses HelloWorldPass::run(Function &F,
                                     FunctionAnalysisManager &AM) {
    errs() << F.getName() << "\n";
    return PreservedAnalyses::all();
}
```

Preserve the results of all other analysis passes

```
class HelloWorldPass : public PassInfoMixin<HelloWorldPass> {
public:
    PreservedAnalyses run(Function &F, FunctionAnalysisManager &AM);
};

} // namespace llvm

#endif // LLVM_TRANSFORMS_HELLOWORLD_H
```

New Pass

<https://llvm.org/docs/WritingAnLLVMNewPMPass.html>

```
PreservedAnalyses HelloWorldPass::run(Function &F,
                                     FunctionAnalysisManager &AM) {
    errs() << F.getName() << "\n";
    return PreservedAnalyses::all();
}
```

Preserve the results of all other analysis passes

```
FUNCTION_PASS("helloworld", HelloWorldPass())
```

llvm/lib/Passes/PassRegistry.def

```
PreservedAnalyses run(Function &F, FunctionAnalysisManager &AM);
};

} // namespace llvm

#endif // LLVM_TRANSFORMS_HELLOWORLD_H
```

Internal Passes

<https://llvm.org/docs/Passes.html>



[LLVM Home](#) | [Documentation](#) » [User Guides](#) » [LLVM's Analysis and Transform Passes](#)

LLVM's Analysis and Transform Passes

- [Introduction](#)
- [Analysis Passes](#)
 - [aa-eval](#): Exhaustive Alias Analysis Precision Evaluator
 - [basic-aa](#): Basic Alias Analysis (stateless AA impl)
 - [basiccg](#): Basic CallGraph Construction
 - [da](#): Dependence Analysis
 - [domfrontier](#): Dominance Frontier Construction
 - [domtree](#): Dominator Tree Construction
 - [dot-callgraph](#): Print Call Graph to "dot" file
 - [dot-cfg](#): Print CFG of function to "dot" file
 - [dot-cfg-only](#): Print CFG of function to "dot" file (with no function bodies)
 - [dot-dom](#): Print dominance tree of function to "dot" file
 - [dot-dom-only](#): Print dominance tree of function to "dot" file (with no function bodies)
 - [dot-post-dom](#): Print postdominance tree of function to "dot" file
 - [dot-post-dom-only](#): Print postdominance tree of function to "dot" file (with no function bodies)

Internal Passes

<https://llvm.org/docs/Passes.html>



[LLVM Home](#) | [Documentation](#) » [User Guides](#) » [LLVM's Analysis and Transform Passes](#)

LLVM's Analysis and Transform Passes

- [Introduction](#)
- [Analysis Passes](#)
 - [aa-eval](#): Exhaustive Alias Analysis Precision Evaluator
 - [basic-aa](#): Basic Alias Analysis (stateless AA impl)
 - [basiccg](#): Basic CallGraph Construction
 - [da](#): Dependence Analysis
 - [domfrontier](#): Dominance Frontier Construction
 - [domtree](#): Dominator Tree Construction
 - [dot-callgraph](#): Print Call Graph to "dot" file
 - [dot-cfg](#): Print CFG of function to "dot" file
 - [dot-cfg-only](#): Print CFG of function to "dot" file (with no function bodies)
 - [dot-dom](#): Print dominance tree of function to "dot" file
 - [dot-dom-only](#): Print dominance tree of function to "dot" file (with no function bodies)
 - [dot-post-dom](#): Print postdominance tree of function to "dot" file
 - [dot-post-dom-only](#): Print postdominance tree of function to "dot" file (with no function bodies)

Internal Passes

<https://llvm.org/docs/Passes.html>



LLVM Home | Documentation » User Guides » LLVM's Analysis and Transform Passes

LLVM's Analysis and Transform Passes

- [Introduction](#)
- [Analysis Passes](#)
 - [aa-eval](#): Exhaustive Alias Analysis Precision Evaluator
 - [basic-aa](#): Basic Alias Analysis (stateless AA impl)
 - [basiccg](#): Basic CallGraph Construction
 - [da](#): Dependence Analysis
 - [domfrontier](#): Dominance Frontier Construction
 - [domtree](#): Dominator Tree Construction
 - [dot-callgraph](#): Print Call Graph to "dot" file
 - [dot-cfg](#): Print CFG of function to "dot" file
 - [dot-cfg-only](#): Print CFG of function to "dot" file (with no function bodies)
 - [dot-dom](#): Print dominance tree of function to "dot" file
 - [dot-dom-only](#): Print dominance tree of function to "dot" file (with no function bodies)
 - [dot-post-dom](#): Print postdominance tree of function to "dot" file
 - [dot-post-dom-only](#): Print postdominance tree of function to "dot" file (with no function bodies)

```
opt --dot-cfg hello.bc
opt --dot-callgraph hello.bc
...
```

LLVM API

- C/C++ API
- <https://llvm.org/docs/ProgrammersManual.html>
- From C/C++ API to Java API
- <https://github.com/bytedeco/javacpp>
- Read the document and send TA emails for questions

PART III: Translation into SSA

Basic Block

- Partition three-address code to basic blocks
 - A basic block is a sequence of consecutive instructions

Basic Block

- Partition three-address code to basic blocks
 - A basic block is a sequence of consecutive instructions
 - The first instruction of each basic block is a leader
 - **[1]** The first instruction of a function is a leader
 - **[2]**
 - **[3]**

Basic Block

- Partition three-address code to basic blocks
 - A basic block is a sequence of consecutive instructions
 - The first instruction of each basic block is a leader
 - **[1]** The first instruction of a function is a leader
 - **[2]** Any instruction that is the target of a jump is a leader
 - **[3]**

Basic Block

- Partition three-address code to basic blocks
 - A basic block is a sequence of consecutive instructions
 - The first instruction of each basic block is a leader
 - **[1]** The first instruction of a function is a leader
 - **[2]** Any instruction that is the target of a jump is a leader
 - **[3]** Any instruction that follows a jump is a leader

Basic Block

```
1)  i = 1
2)  j = 1
3)  t1 = 10 * i
4)  t2 = t1 + j
5)  t3 = 8 * t2
6)  t4 = t3 - 88
7)  a[t4] = 0.0
8)  j = j + 1
9)  if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Basic Block

```
1)  i = 1
2)  j = 1
3)  t1 = 10 * i
4)  t2 = t1 + j
5)  t3 = 8 * t2
6)  t4 = t3 - 88
7)  a[t4] = 0.0
8)  j = j + 1
9)  if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Basic Block

1) i = 1
2) j = 1

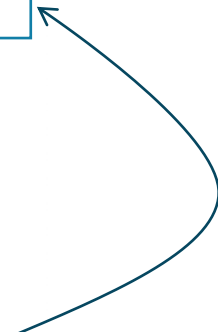
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)



Basic Block

1) i = 1
2) j = 1
3) t1 = 10 * i

4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)



Basic Block

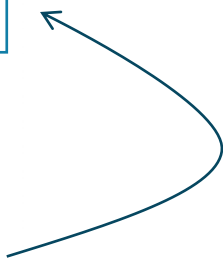
1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)

Basic Block

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)

Basic Block

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



Basic Block

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)

Basic Block

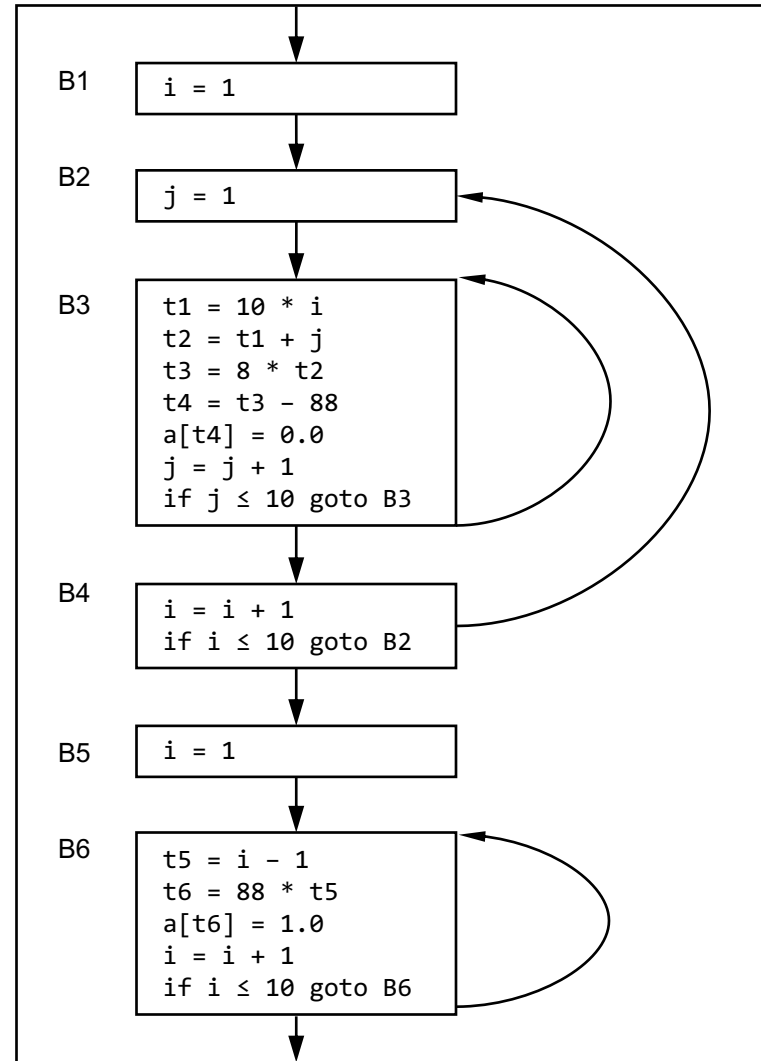
1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)

Basic Block

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)

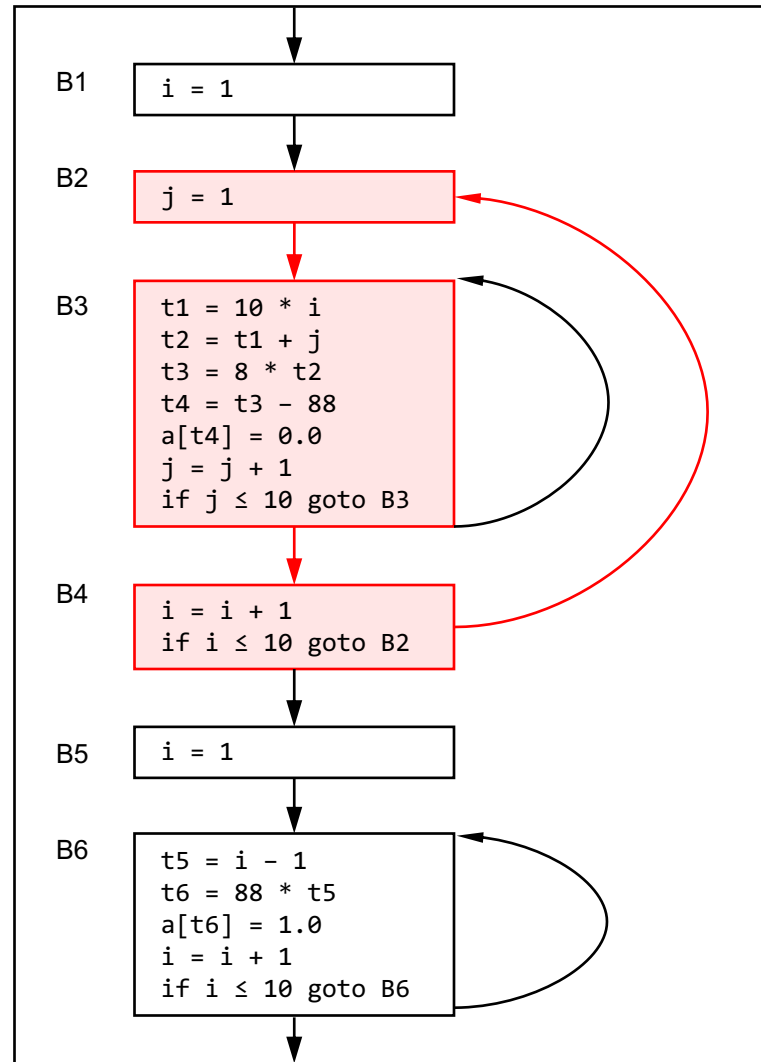
Control Flow Graph

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



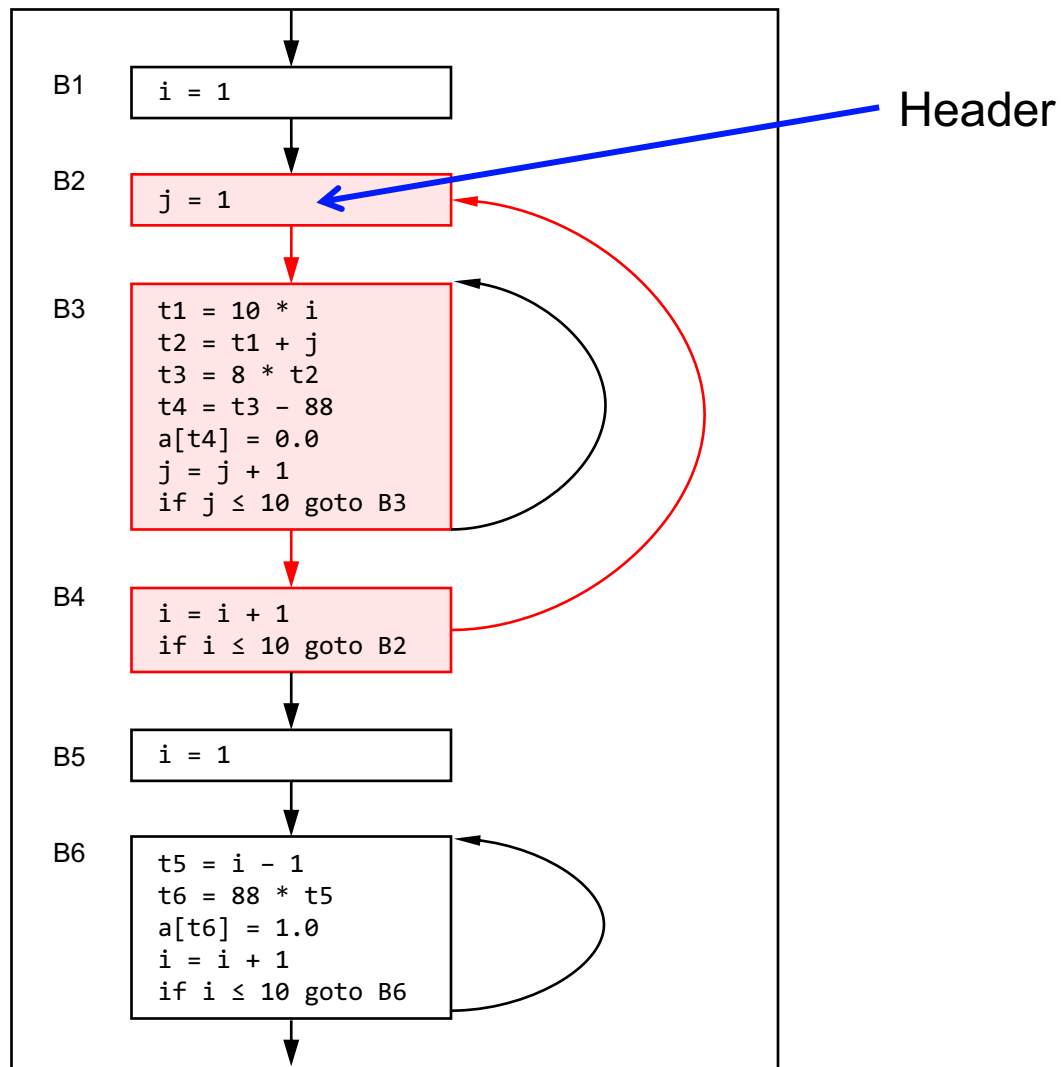
Loop

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



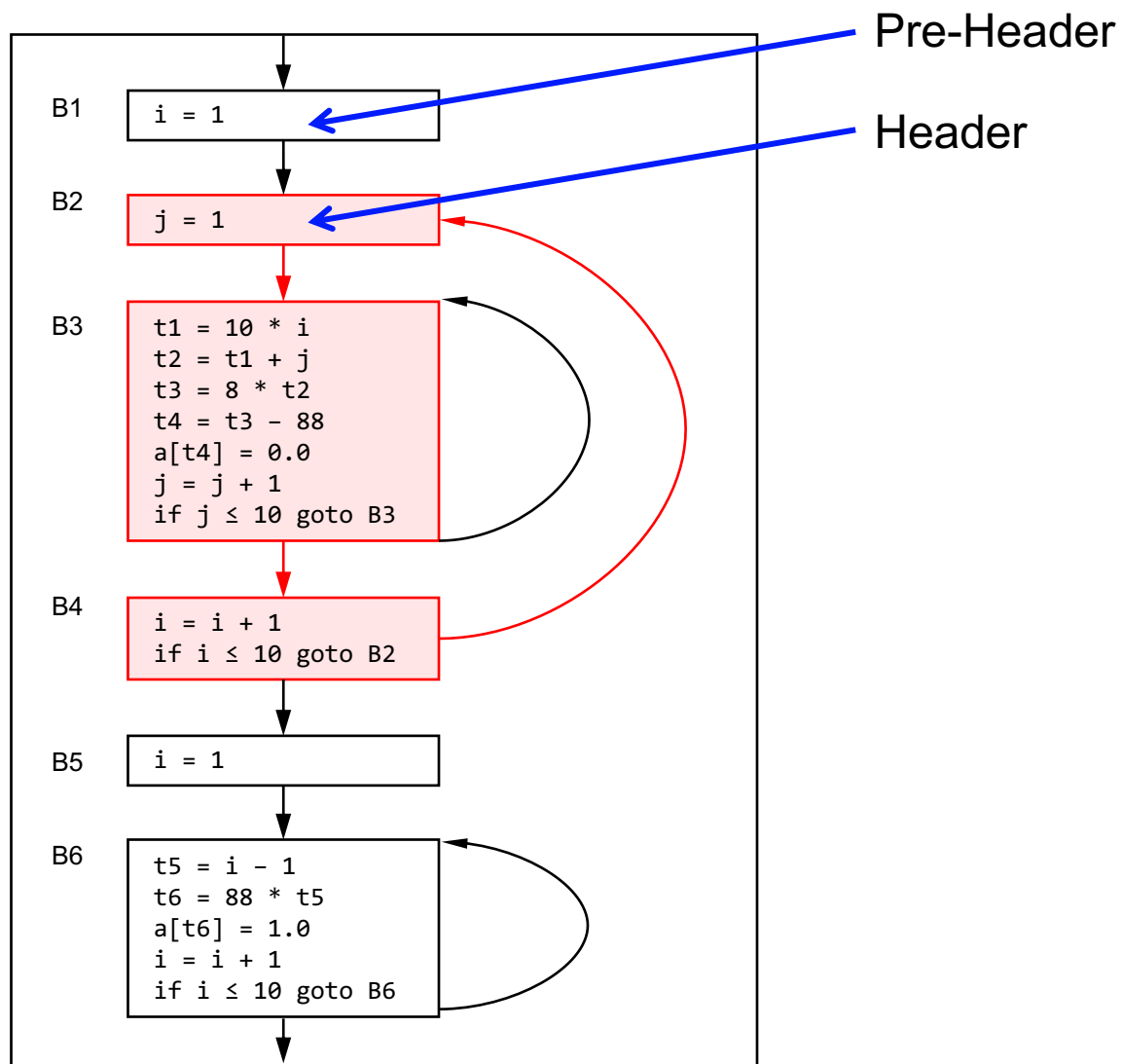
Loop

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



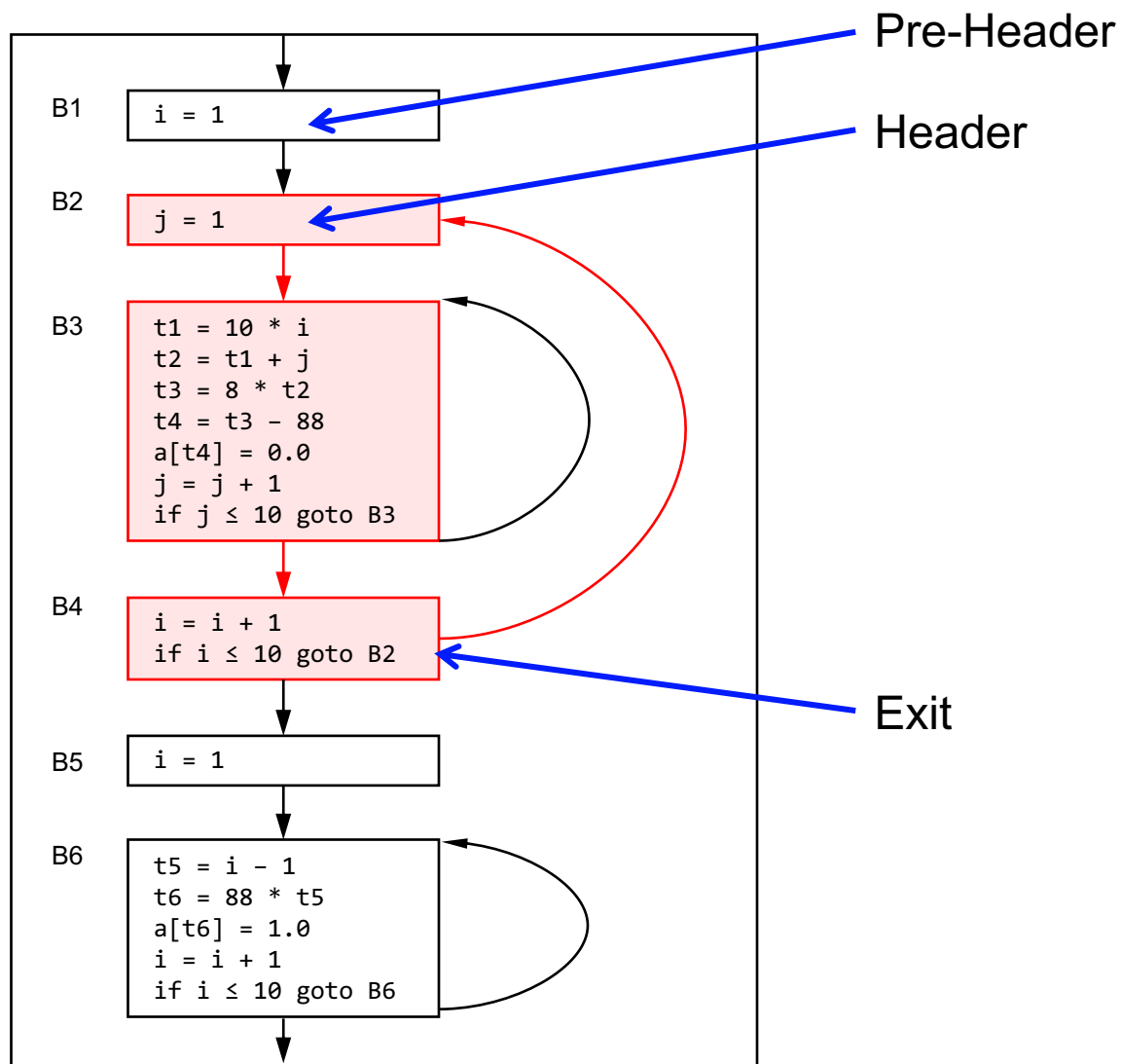
Loop

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



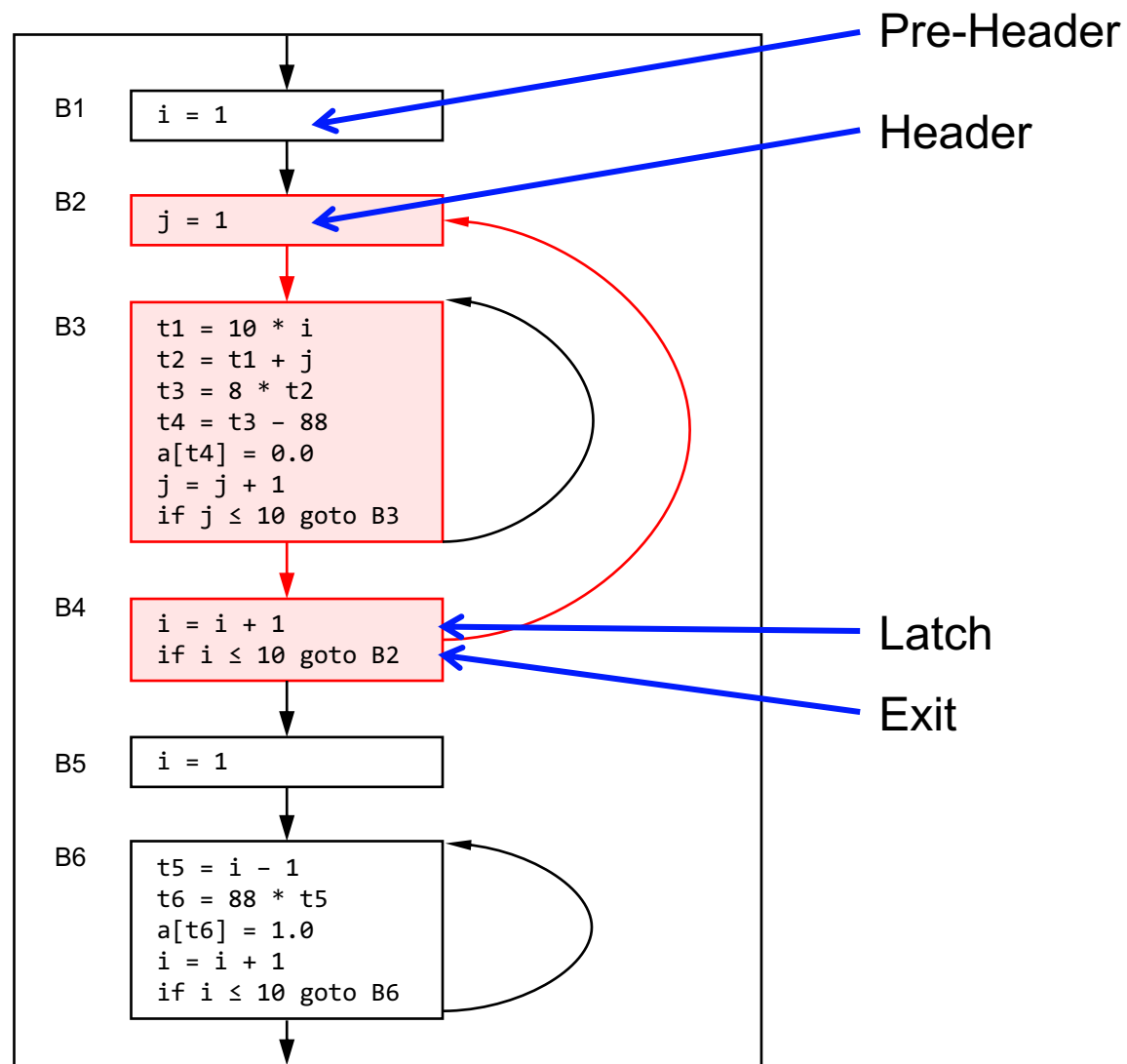
Loop

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



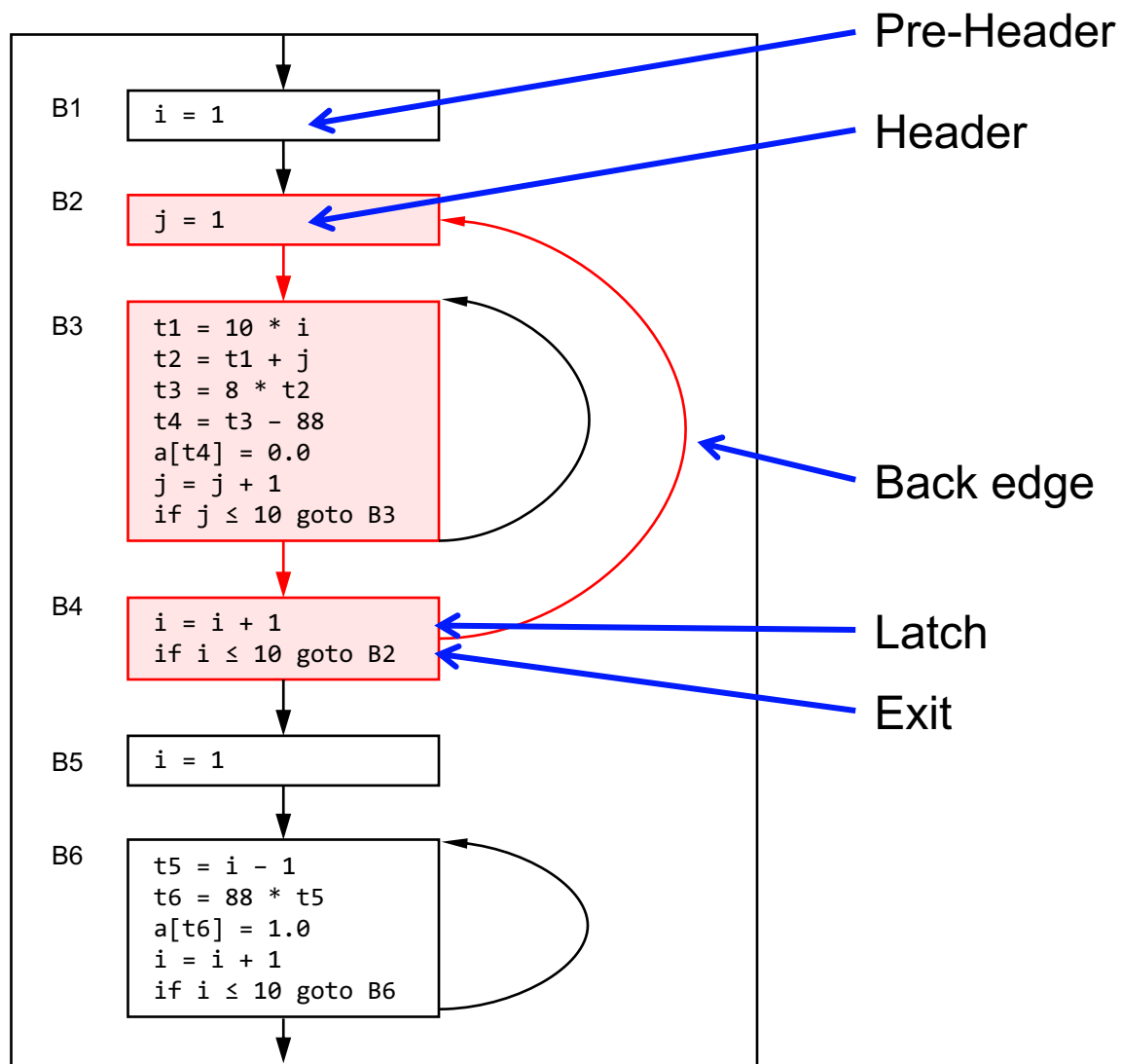
Loop

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



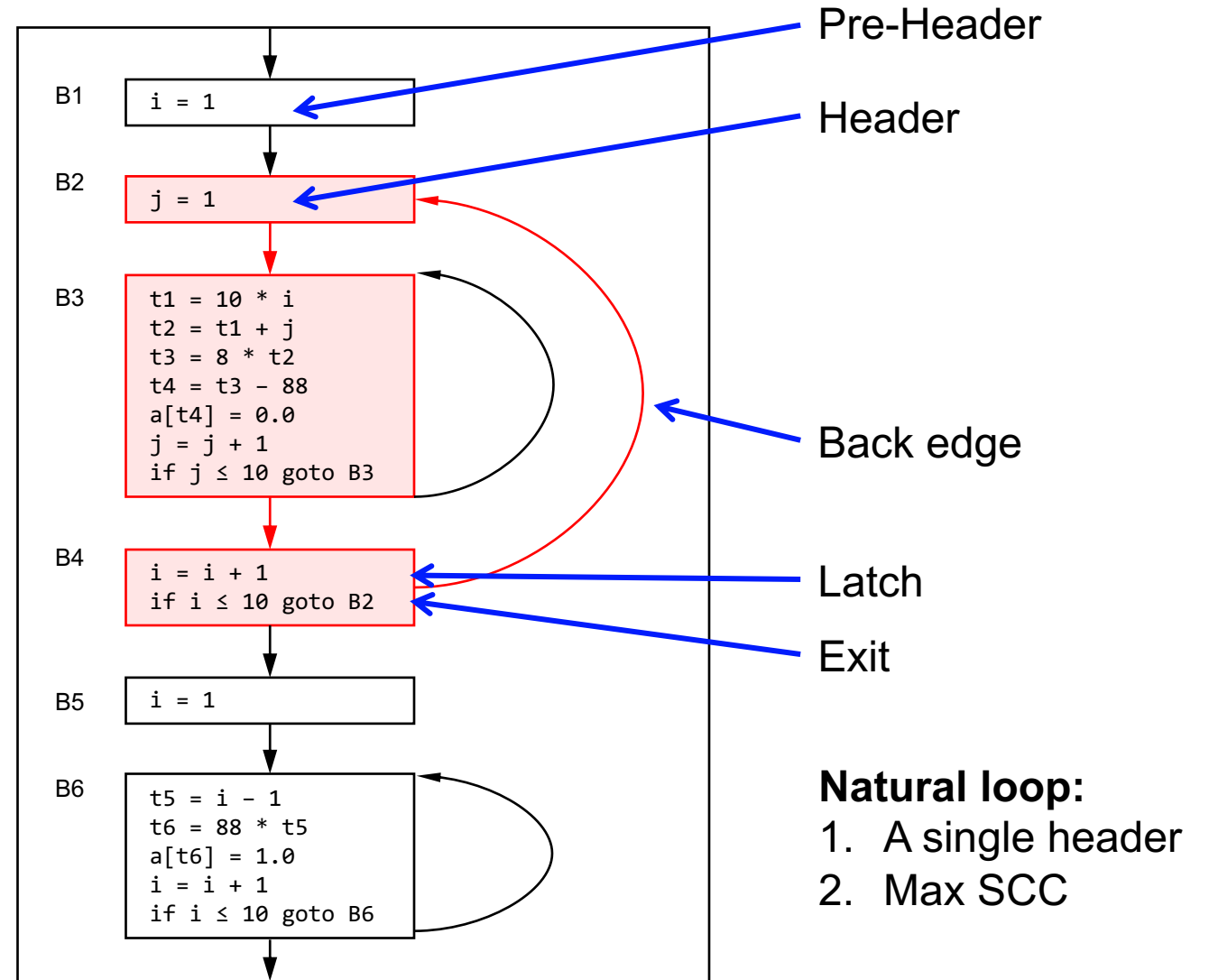
Loop

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



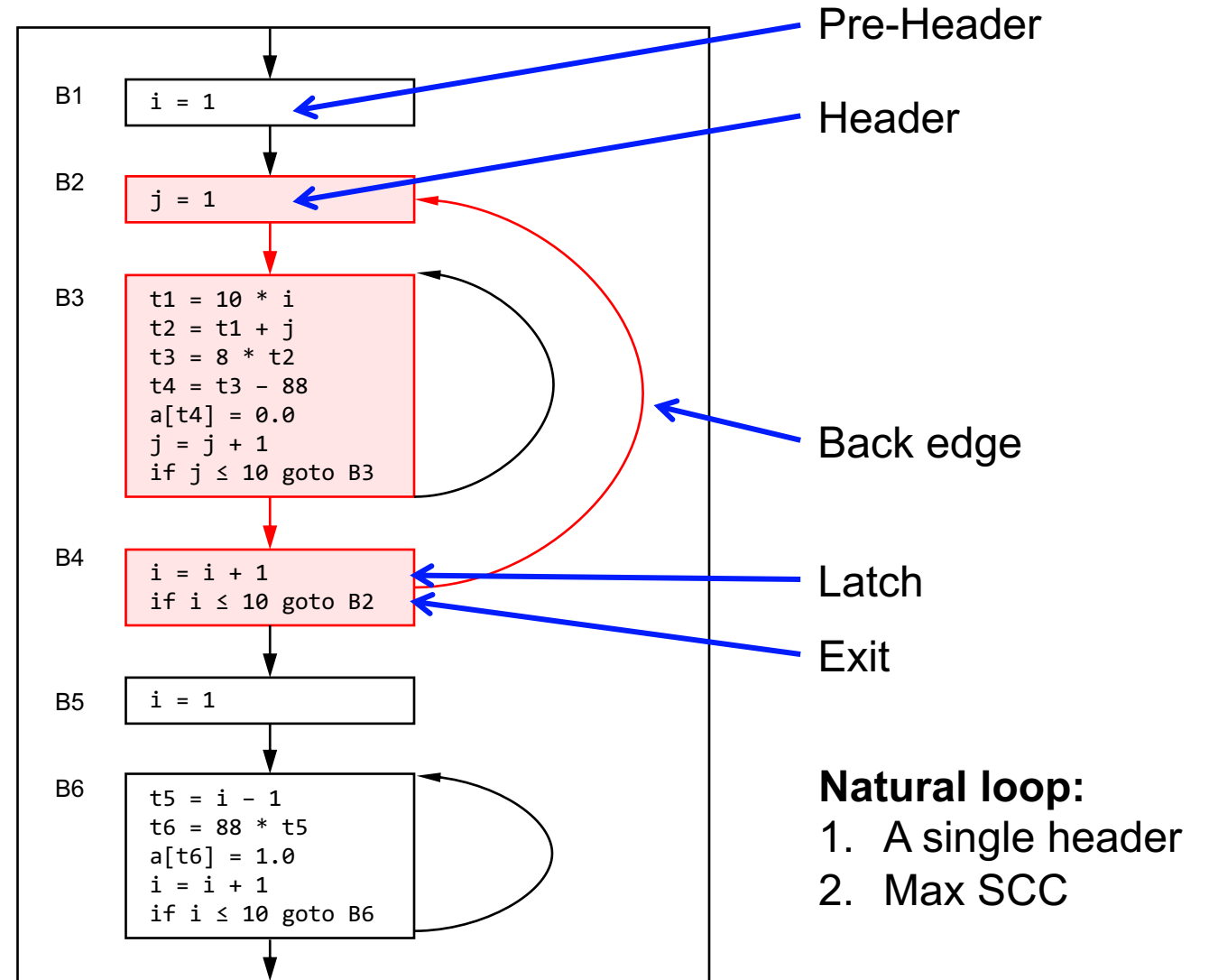
Loop

1)	i = 1
2)	j = 1
3)	t1 = 10 * i
4)	t2 = t1 + j
5)	t3 = 8 * t2
6)	t4 = t3 - 88
7)	a[t4] = 0.0
8)	j = j + 1
9)	if j <= 10 goto (3)
10)	i = i + 1
11)	if i <= 10 goto (2)
12)	i = 1
13)	t5 = i - 1
14)	t6 = 88 * t5
15)	a[t6] = 1.0
16)	i = i + 1
17)	if i <= 10 goto (13)



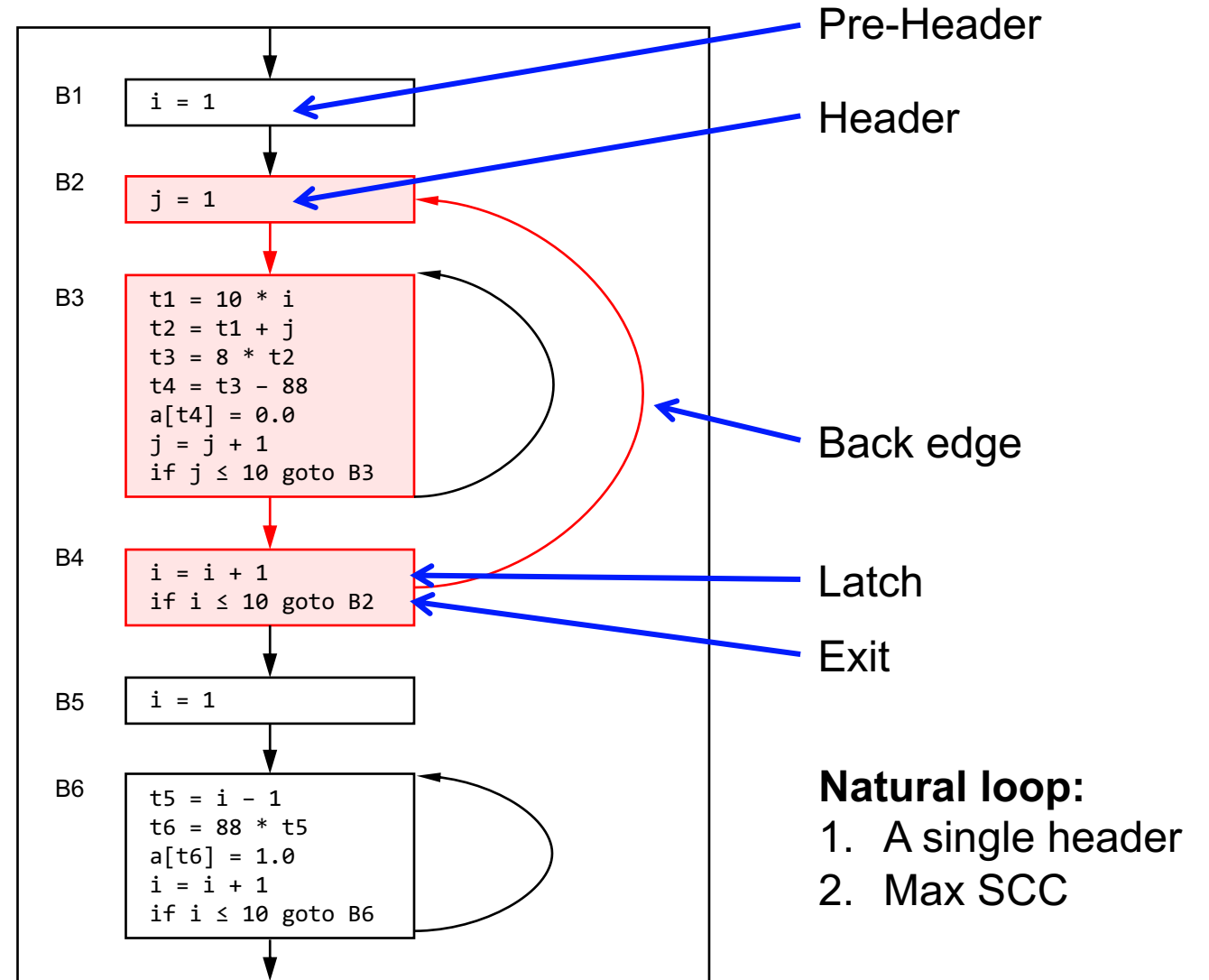
Dominance

- What does that mean for a single header?



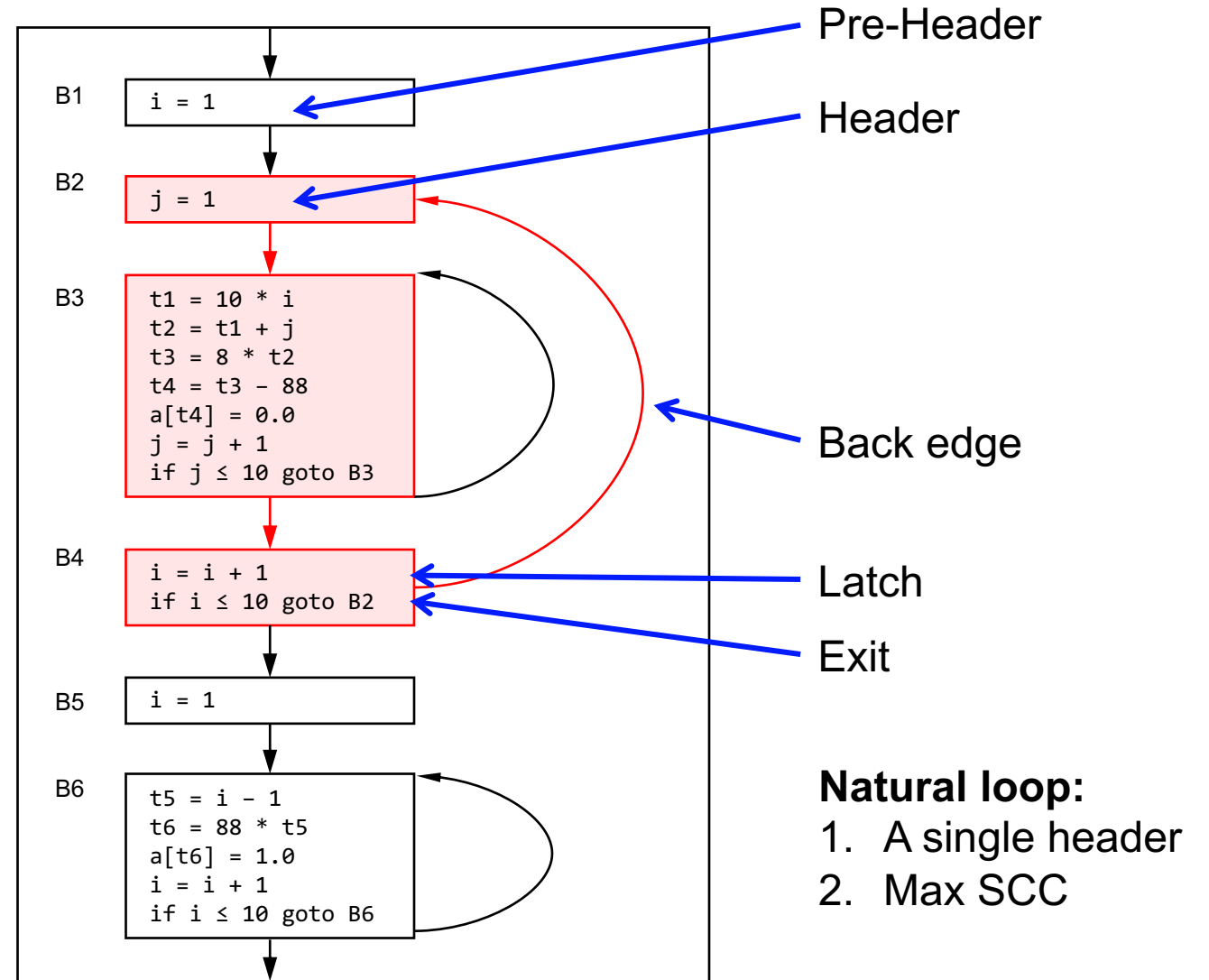
Dominance

- What does that mean for a single header?
- The header dominates all blocks in the loop.



Dominance

- What does that mean for a single header?
- The header dominates all blocks in the loop.
 - A **dom** B
 - if all paths from Entry to B goes through A



Post-Dominance

- **A dom B**
 - if all paths from Entry to B goes through A
- **A post-dom B**
 - if all paths from B to Exit goes through A

Post-Dominance

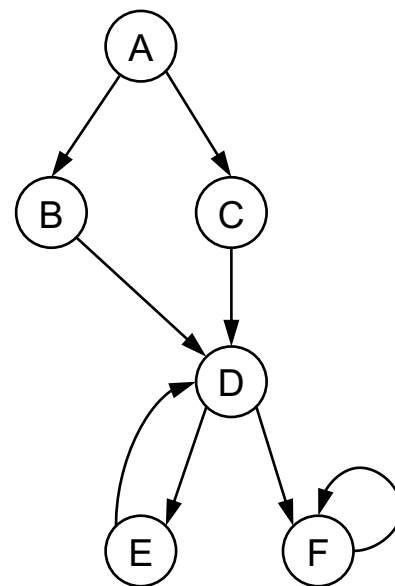
- **A dom B**
 - if all paths from Entry to B goes through A
- **A post-dom B**
 - if all paths from B to Exit goes through A
- **Strict** (post-)dominance – A (post-)dom B but $A \neq B$

Post-Dominance

- **A dom B**
 - if all paths from Entry to B goes through A
- **A post-dom B**
 - if all paths from B to Exit goes through A
- **Strict** (post-)dominance – A (post-)dom B but $A \neq B$
- **Immediate** dominance – A strict-dom B, but there's no C, such that A strict-dom C, C strict-dom B

Dominator Tree

- Almost linear time to build a dominator tree.
 - Node: Block
 - Edge: Immediate dom relation

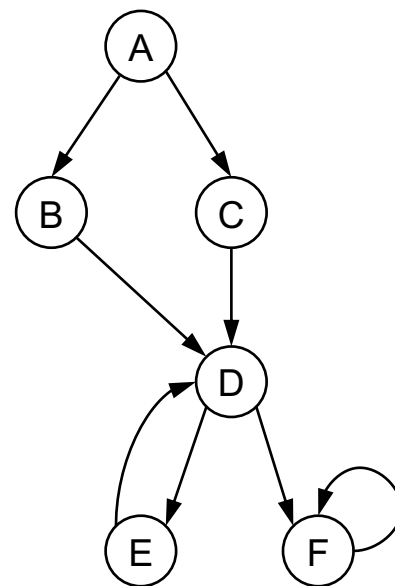


Flow Graph

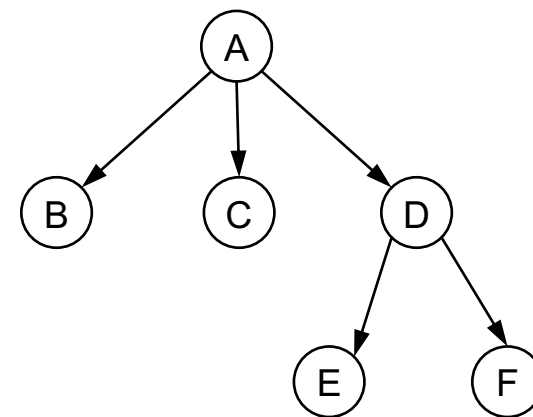
Try!

Dominator Tree

- Almost linear time to build a dominator tree.
 - Node: Block
 - Edge: Immediate dom relation



Flow Graph



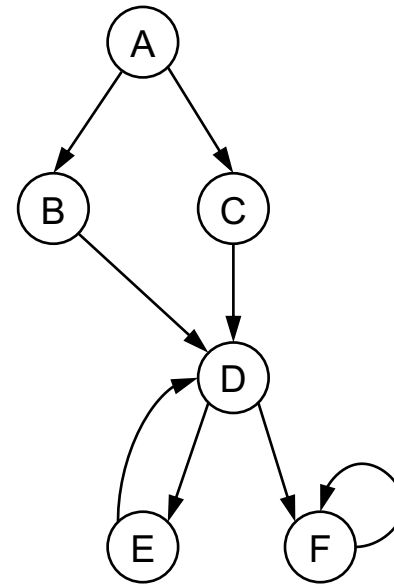
Dominator Tree

Dominator Tree

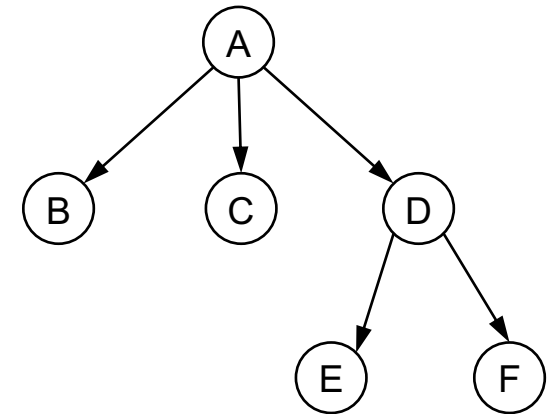
- Almost linear time to build a dominator tree.

- Node: Block
- Edge: Immediate dom relation

- (Nearest) common dominator
 - LCA – lowest common ancestor
 - Almost linear preprocessing time
 - Constant query time



Flow Graph



Dominator Tree

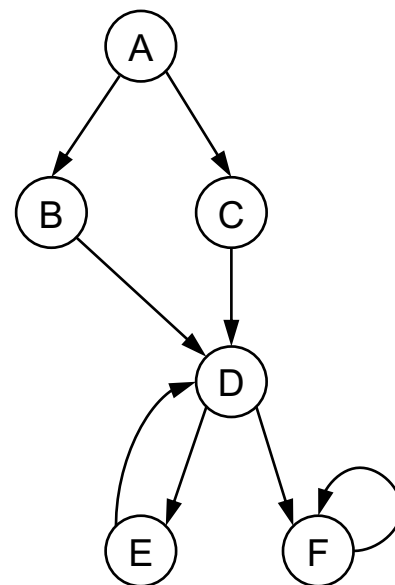
Dominator Tree

- Almost linear time to build a dominator tree.

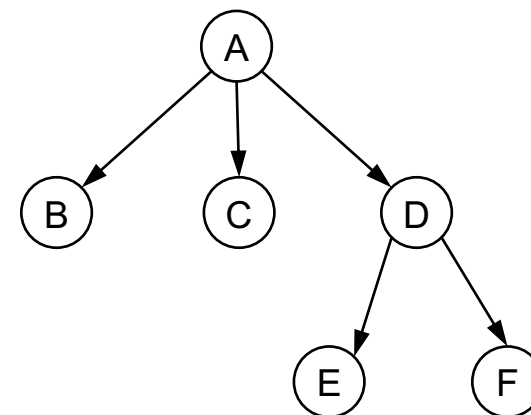
- Node: Block
- Edge: Immediate dom relation

- (Nearest) common dominator

- LCA – lowest common ancestor
- Almost linear preprocessing time
- Constant query time
- **Application:** placing variable definition



Flow Graph



Dominator Tree

Dominance vs. Ctrl Dependence

Dominance vs. Ctrl Dependence

- Block B is control dependent on Block A if and only if
 - The execution result of A determines if B will be executed

Dominance vs. Ctrl Dependence

- Block B is control dependent on Block A if and only if
 - The execution result of A determines if B will be executed
- Block B is control dependent on Block A if and only if
 - A has multiple successors
 - Not all successors can reach B

Dominance vs. Ctrl Dependence

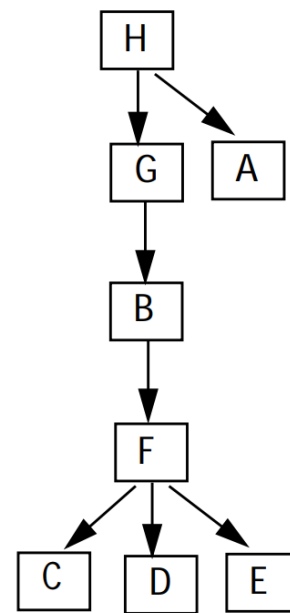
- Block B is control dependent on Block A if and only if
 - The execution result of A determines if B will be executed
- Block B is control dependent on Block A if and only if
 - A has multiple successors
 - Not all successors can reach B
- By definition, we always need a forward traversal from all successors of B to test if A ctrl-depends on B.

Dominance vs. Ctrl Dependence

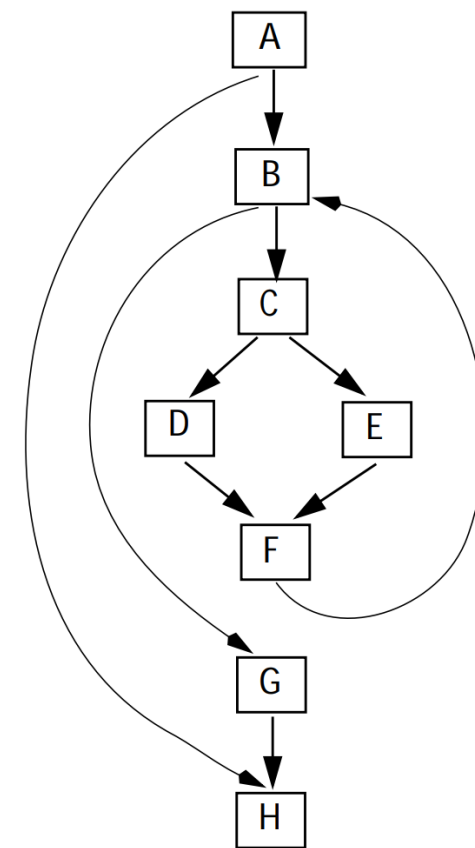
- Block B is control dependent on Block A if and only if
 - The execution result of A determines if B will be executed
- Block B is control dependent on Block A if and only if
 - A has multiple successors
 - Not all successors can reach B
- By definition, we always need a forward traversal from all successors of B to test if A ctrl-depends on B. **Too expensive!!**

Dominance vs. Ctrl Dependence

- Block B is control dependent on Block A if and only if
 - B post-dominates a successor of A
 - B does not post-dominates all successors of A



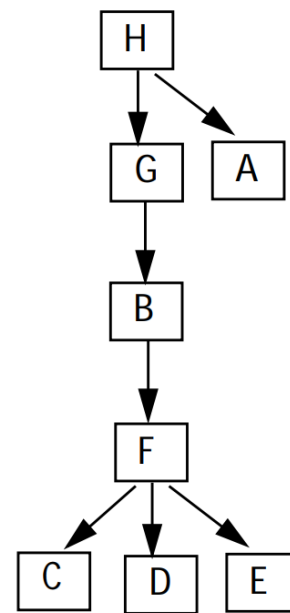
Postominator Tree



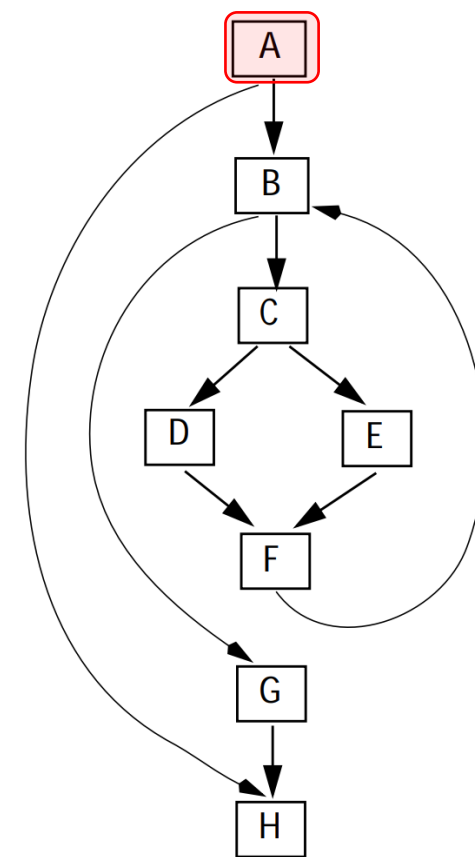
Control Flow Graph

Dominance vs. Ctrl Dependence

- Block B is control dependent on Block A if and only if
 - B post-dominates a successor of A
 - B does not post-dominates all successors of A



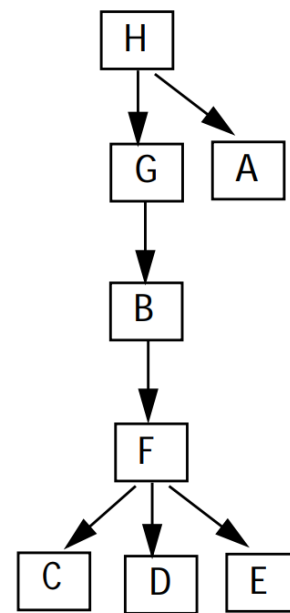
Postominator Tree



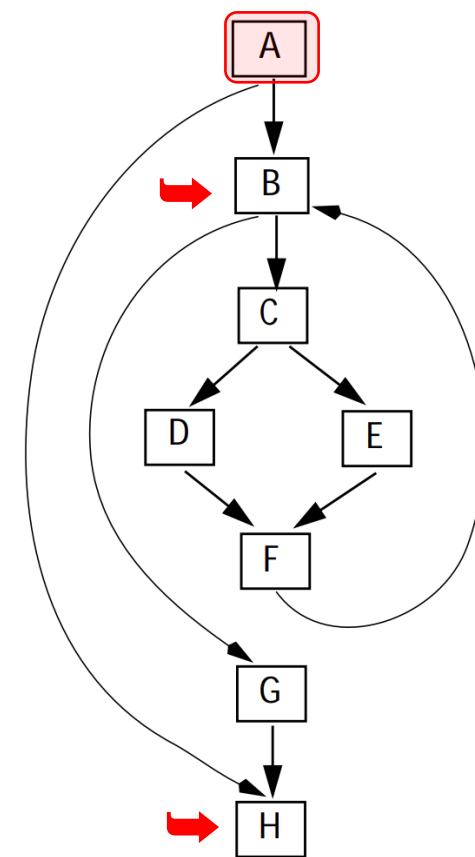
Control Flow Graph

Dominance vs. Ctrl Dependence

- Block B is control dependent on Block A if and only if
 - B post-dominates a successor of A
 - B does not post-dominates all successors of A



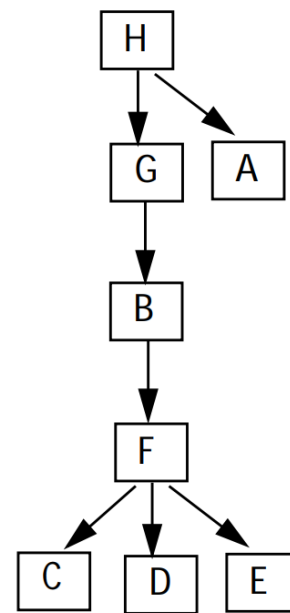
Postominator Tree



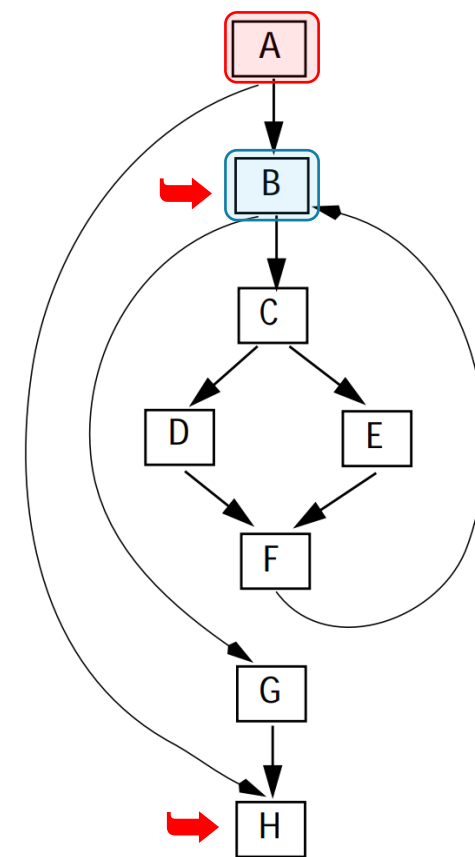
Control Flow Graph

Dominance vs. Ctrl Dependence

- Block B is control dependent on Block A if and only if
 - B post-dominates a successor of A
 - B does not post-dominates all successors of A



Postominator Tree

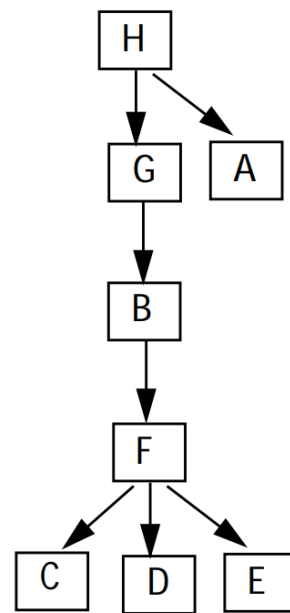


Control Flow Graph

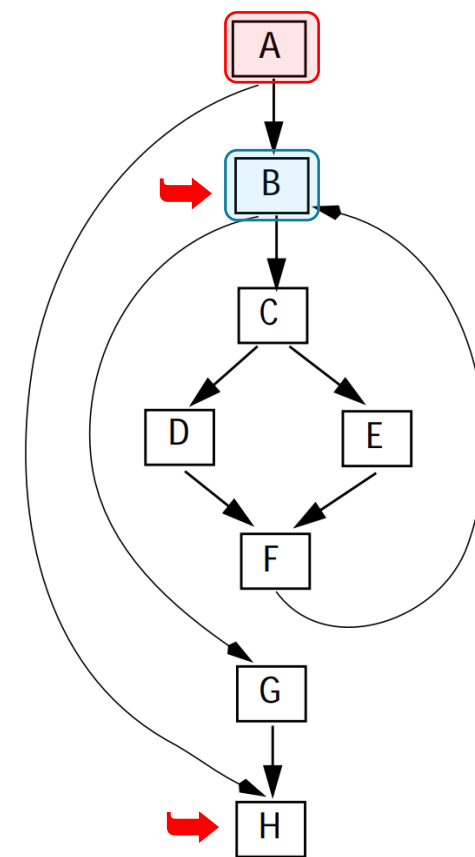
Dominance vs. Ctrl Dependence

- Block B is control dependent on Block A if and only if
 - B post-dominates a successor of A
 - B does not post-dominates all successors of A

Try!



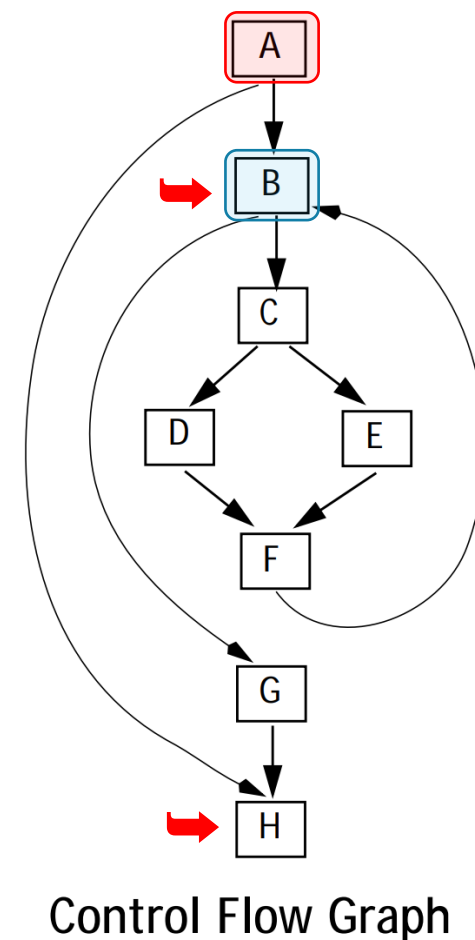
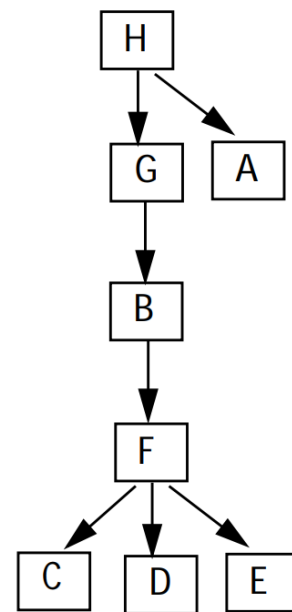
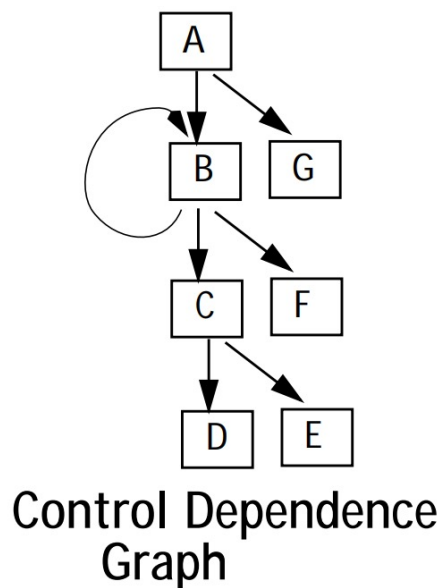
Postominator Tree



Control Flow Graph

Dominance vs. Ctrl Dependence

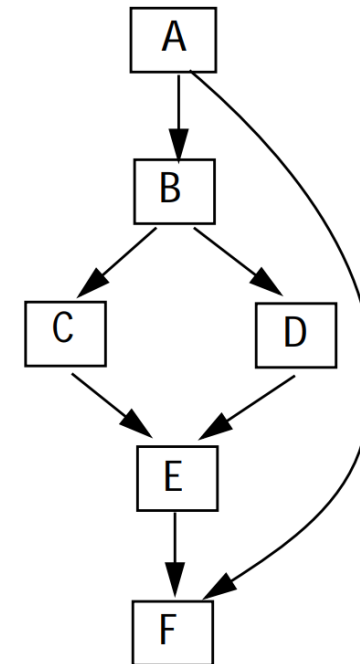
- Block B is control dependent on Block A if and only if
 - B post-dominates a successor of A
 - B does not post-dominates all successors of A



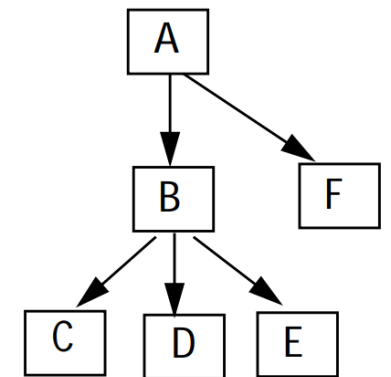
Dominance Frontier

- $DF(B) = \{ \dots \}$
 - The immediate successors of the blocks dominated by B
 - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier						



Control Flow Graph

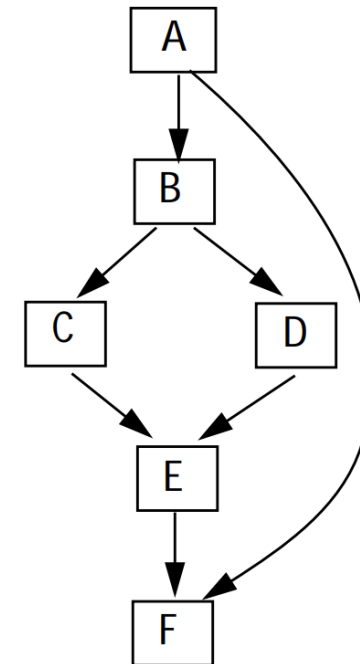


Dominator Tree

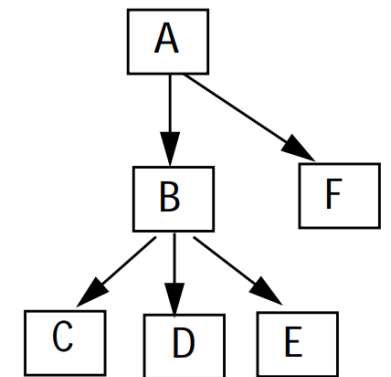
Dominance Frontier

- $DF(B) = \{ \dots \}$
 - The immediate successors of the blocks dominated by B
 - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier	{ }					



Control Flow Graph

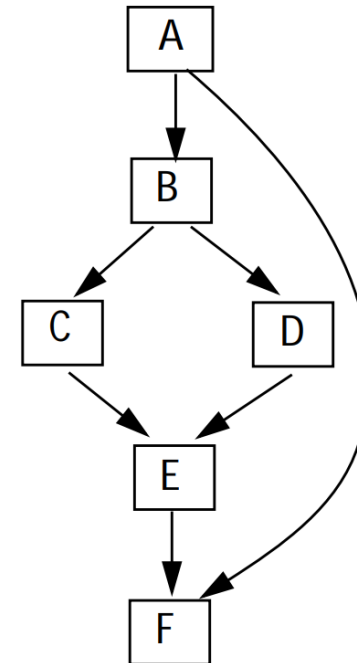


Dominator Tree

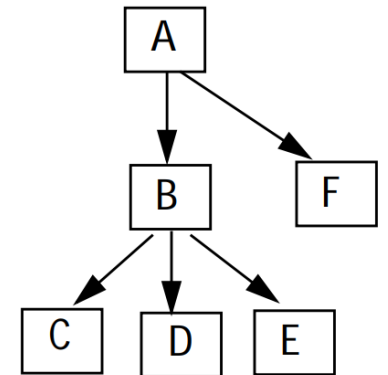
Dominance Frontier

- $DF(B) = \{ \dots \}$
 - The immediate successors of the blocks dominated by B
 - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}				



Control Flow Graph

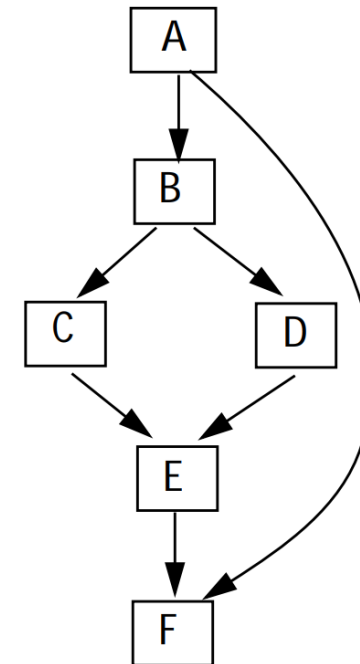


Dominator Tree

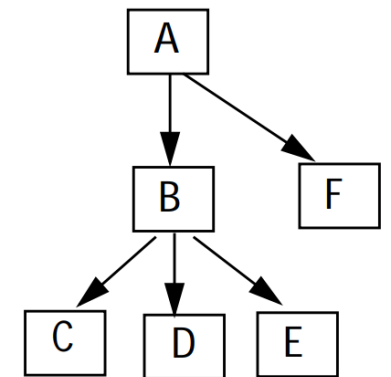
Dominance Frontier

- $DF(B) = \{ \dots \}$
 - The immediate successors of the blocks dominated by B
 - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}	{E}			



Control Flow Graph

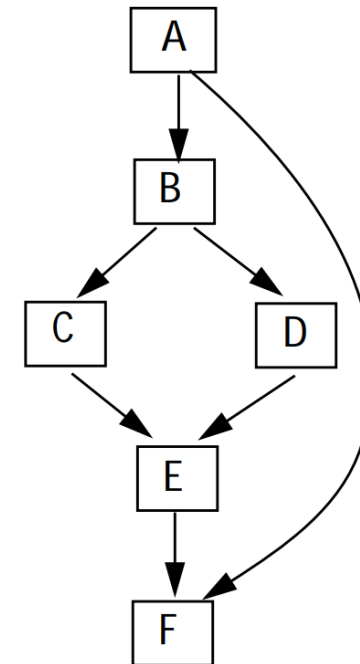


Dominator Tree

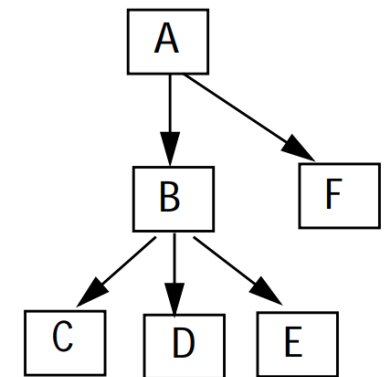
Dominance Frontier

- $DF(B) = \{ \dots \}$
 - The immediate successors of the blocks dominated by B
 - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}	{E}	{E}		



Control Flow Graph

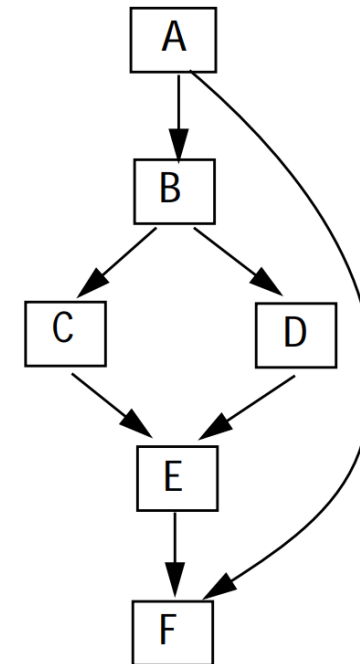


Dominator Tree

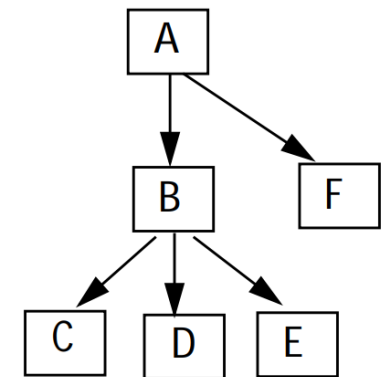
Dominance Frontier

- $DF(B) = \{ \dots \}$
 - The immediate successors of the blocks dominated by B
 - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}	{E}	{E}	{F}	



Control Flow Graph

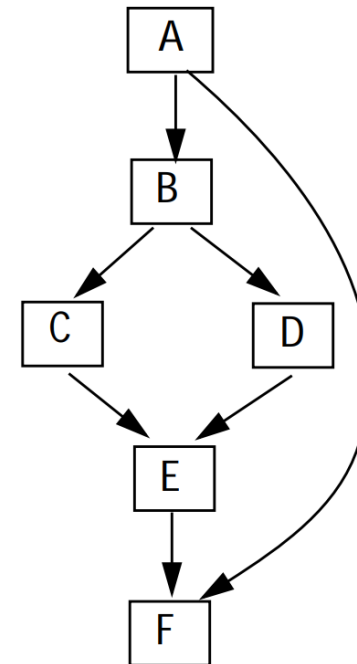


Dominator Tree

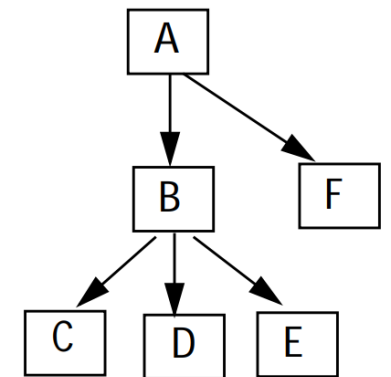
Dominance Frontier

- $DF(B) = \{ \dots \}$
 - The immediate successors of the blocks dominated by B
 - Not strictly dominated by B

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}	{E}	{E}	{F}	{ }



Control Flow Graph



Dominator Tree

Dominance Frontier

- Extend the definition of DF to a set of blocks
 - $DF(\mathbf{Bset}) = \bigcup_{B \in \mathbf{Bset}} DF(B)$

Iterated Dominance Frontier (IDF)

- Iterated DF of Bset

Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
 - $DF_1 = DF(Bset);$

Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
 - $DF_1 = DF(Bset); Bset = Bset \cup DF_1$

Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
 - $DF_1 = DF(Bset); \text{ Bset} = \text{Bset} \cup DF_1$
 - $DF_2 = DF(Bset);$

Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
 - $DF_1 = DF(Bset); Bset = Bset \cup DF_1$
 - $DF_2 = DF(Bset); Bset = Bset \cup DF_2$

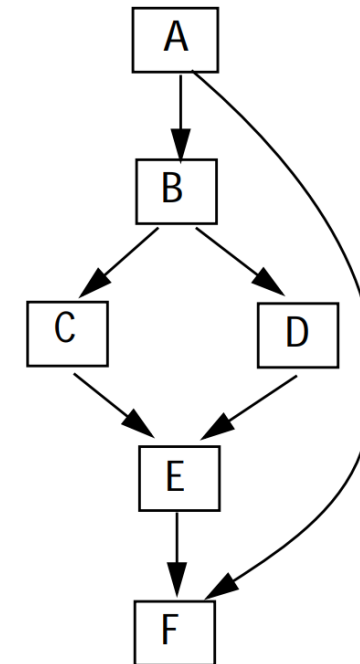
Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
 - $DF_1 = DF(Bset); Bset = Bset \cup DF_1$
 - $DF_2 = DF(Bset); Bset = Bset \cup DF_2$
 -
 - until fixed point! (i.e., $DF_n = DF_{n-1}$)

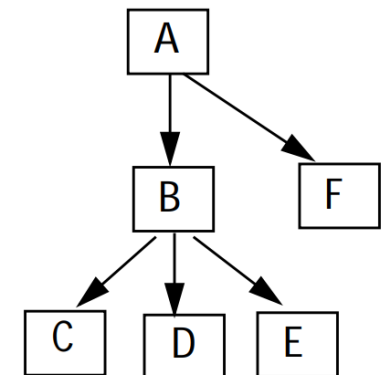
Iterated Dominance Frontier (IDF)

- Iterated DF of Bset
 - $DF_1 = DF(\text{Bset}); \text{Bset} = \text{Bset} \cup DF_1$
 - $DF_2 = DF(\text{Bset}); \text{Bset} = \text{Bset} \cup DF_2$
 -
 - until fixed point! (i.e., $DF_n = DF_{n-1}$)
- $IDF(\{A, B, C\}) = \{ ? \}$

Block	A	B	C	D	E	F
Dominance Frontier	{ }	{F}	{E}	{E}	{F}	{ }



Control Flow Graph



Dominator Tree

IDF vs. SSA

- What is SSA?

IDF vs. SSA

- What is SSA?
- Where should we insert ϕ to translate IR to SSA?

IDF vs. SSA

- Where should we insert ϕ to translate IR to SSA?
- By definition, merge point of multiple paths for a variable

```
if ( flag ) x = -1; else x = 1;  
y = x * a;
```

```
if ( flag ) x1 = -1; else x2 = 1;  
x3 =  $\phi$  (x1, x2);  
y = x3 * a
```

IDF vs. SSA

- Where should we insert ϕ to translate IR to SSA?
- By definition, merge point of multiple paths for a variable
 - **Too complex, path explosion**

```
if ( flag ) x = -1; else x = 1;
y = x * a;
```

```
if ( flag ) x1 = -1; else x2 = 1;
x3 =  $\phi$  (x1, x2);
y = x3 * a
```

IDF vs. SSA

- Where should we insert ϕ to translate IR to SSA?
- By definition, merge point of multiple paths for a variable
- **The merge point of multiple definitions of a variable**

```
if ( flag ) x = -1; else x = 1;  
y = x * a;
```

```
if ( flag ) x1 = -1; else x2 = 1;  
x3 =  $\phi$  (x1, x2);  
y = x3 * a
```

IDF vs. SSA

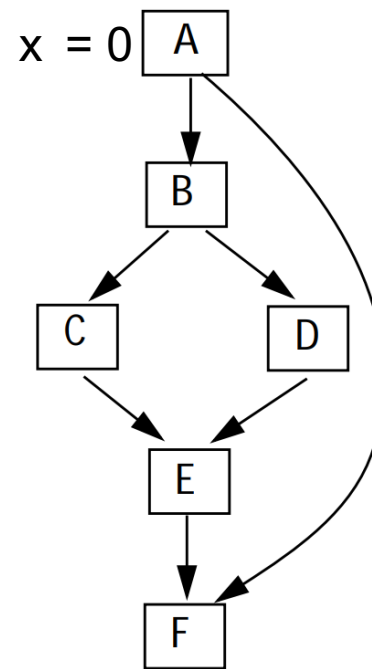
- Definition of a variable can be passed to blocks

IDF vs. SSA

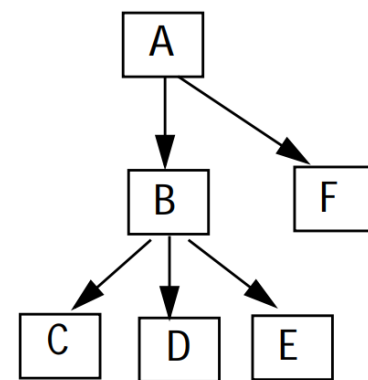
- Definition of a variable can be passed to blocks
 - dominated by the definition or

IDF vs. SSA

- Definition of a variable can be passed to blocks
 - dominated by the definition or



Control Flow Graph

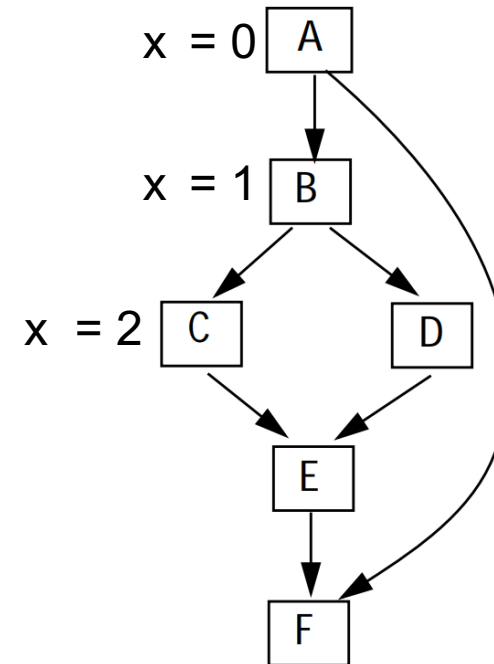


Dominator Tree

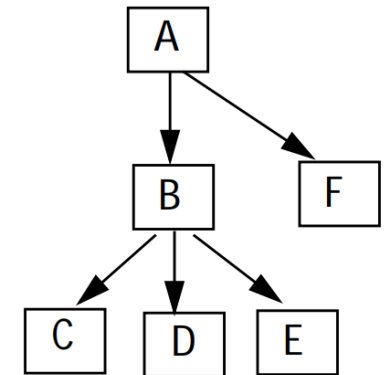
IDF vs. SSA

- Definition of a variable can be passed to blocks
 - dominated by the definition or
 - in the IDF, *e.g.*, $IDF(\{A, B, C\}) = \{E, F\}$

Block	A	B	C	D	E	F
Dominance Frontier	{}	{F}	{E}	{E}	{F}	{}



Control Flow Graph

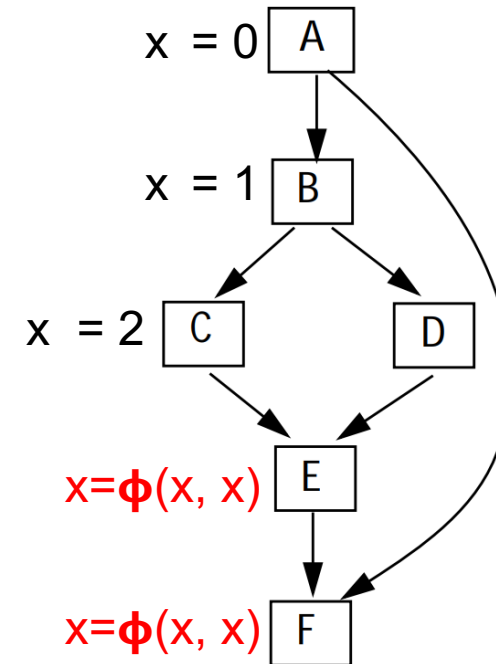


Dominator Tree

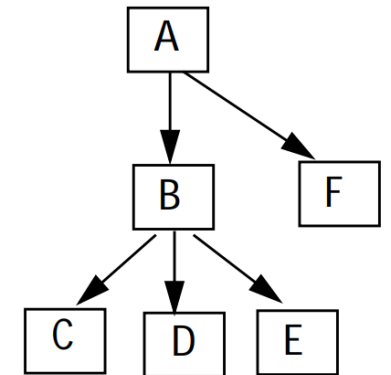
IDF vs. SSA

- Definition of a variable can be passed to blocks
 - dominated by the definition or
 - in the IDF, *e.g.*, $IDF(\{A, B, C\}) = \{E, F\}$
 - where ϕ are inserted

Block	A	B	C	D	E	F
Dominance Frontier	{}	{F}	{E}	{E}	{F}	{}



Control Flow Graph

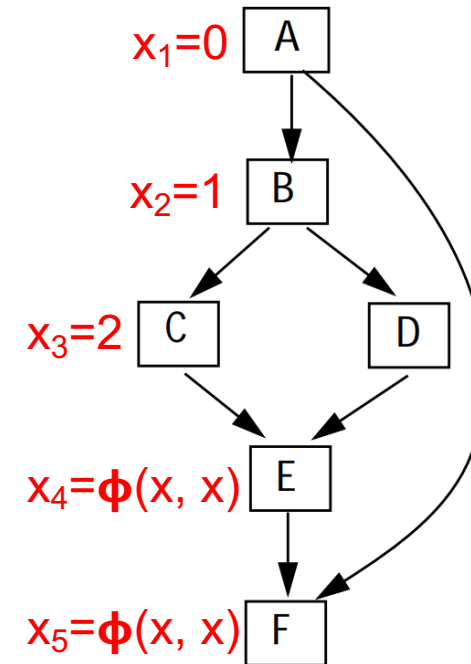


Dominator Tree

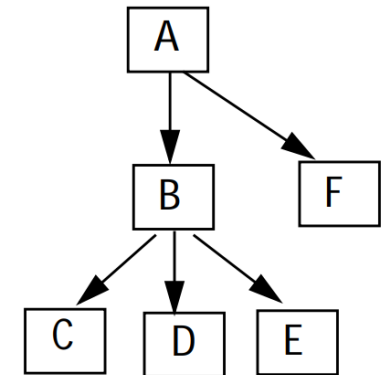
IDF vs. SSA

- Definition of a variable can be passed to blocks
 - dominated by the definition or
 - in the IDF, *e.g.*, $IDF(\{A, B, C\}) = \{E, F\}$
 - where ϕ are inserted

Block	A	B	C	D	E	F
Dominance Frontier	{}	{F}	{E}	{E}	{F}	{}



Control Flow Graph



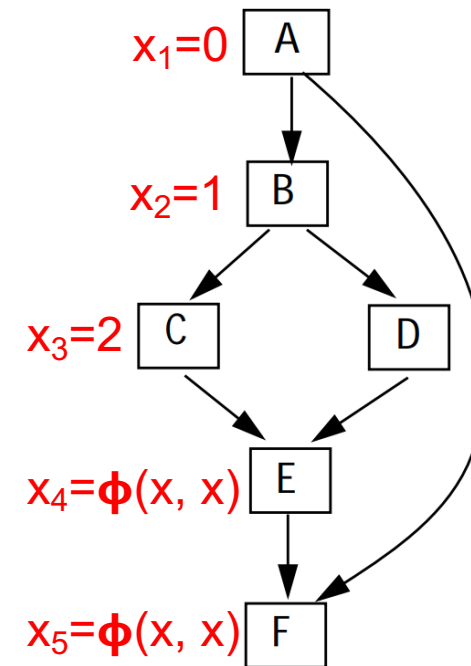
Dominator Tree

IDF vs. SSA

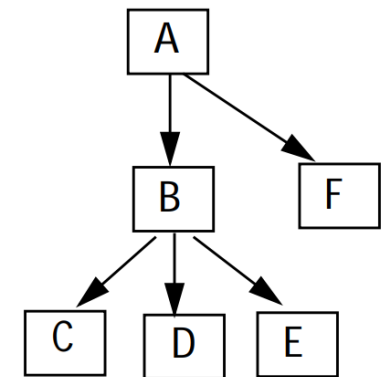
- Definition of a variable can be passed to blocks
 - dominated by the definition or
 - in the IDF, *e.g.*, $IDF(\{A, B, C\}) = \{E, F\}$
 - where ϕ are inserted

Block	A	B	C	D	E	F
Dominance Frontier	{}	{F}	{E}	{E}	{F}	{}

- find proper definitions by climbing DT



Control Flow Graph



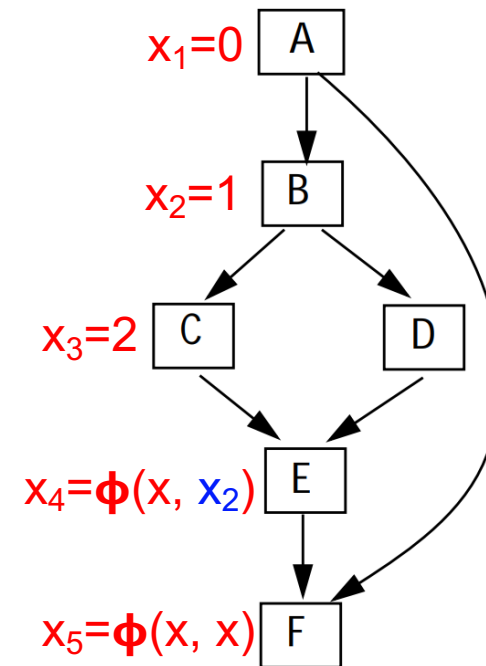
Dominator Tree

IDF vs. SSA

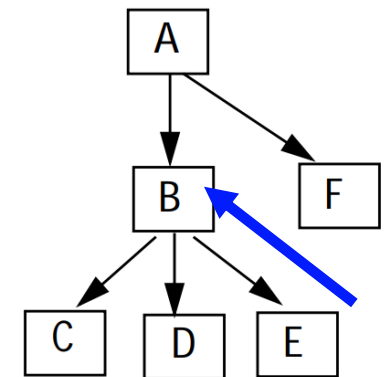
- Definition of a variable can be passed to blocks
 - dominated by the definition or
 - in the IDF, *e.g.*, $IDF(\{A, B, C\}) = \{E, F\}$
 - where ϕ are inserted

Block	A	B	C	D	E	F
Dominance Frontier	{}	{F}	{E}	{E}	{F}	{}

- find proper definitions by climbing DT



Control Flow Graph



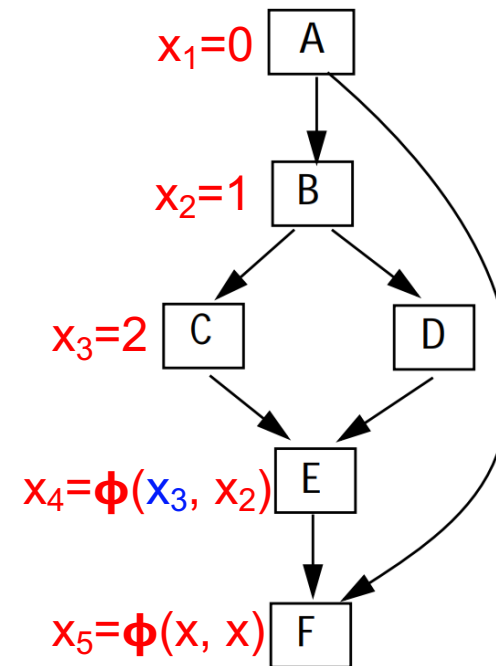
Dominator Tree

IDF vs. SSA

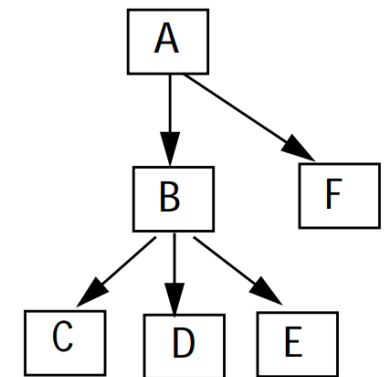
- Definition of a variable can be passed to blocks
 - dominated by the definition or
 - in the IDF, *e.g.*, $IDF(\{A, B, C\}) = \{E, F\}$
 - where ϕ are inserted

Block	A	B	C	D	E	F
Dominance Frontier	{}	{F}	{E}	{E}	{F}	{}

- find proper definitions by climbing DT



Control Flow Graph



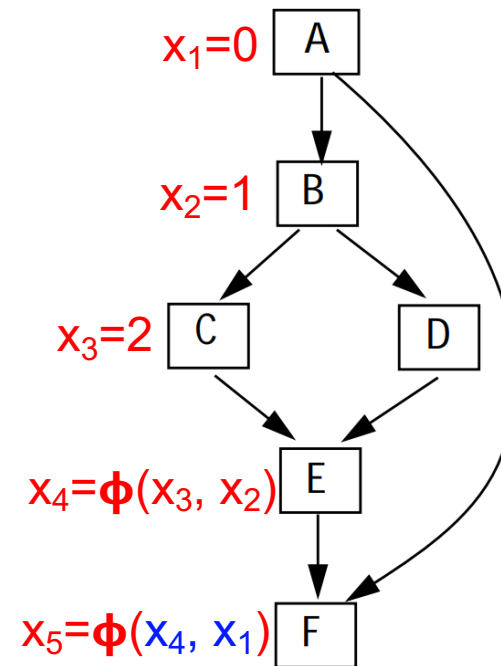
Dominator Tree

IDF vs. SSA

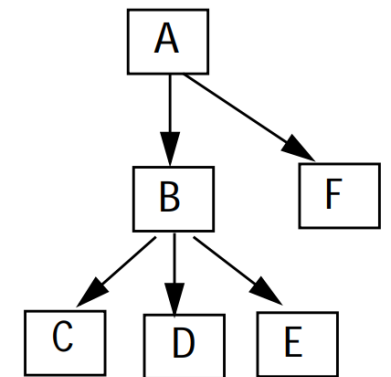
- Definition of a variable can be passed to blocks
 - dominated by the definition or
 - in the IDF, *e.g.*, $IDF(\{A, B, C\}) = \{E, F\}$
 - where ϕ are inserted

Block	A	B	C	D	E	F
Dominance Frontier	{}	{F}	{E}	{E}	{F}	{}

- find proper definitions by climbing DT

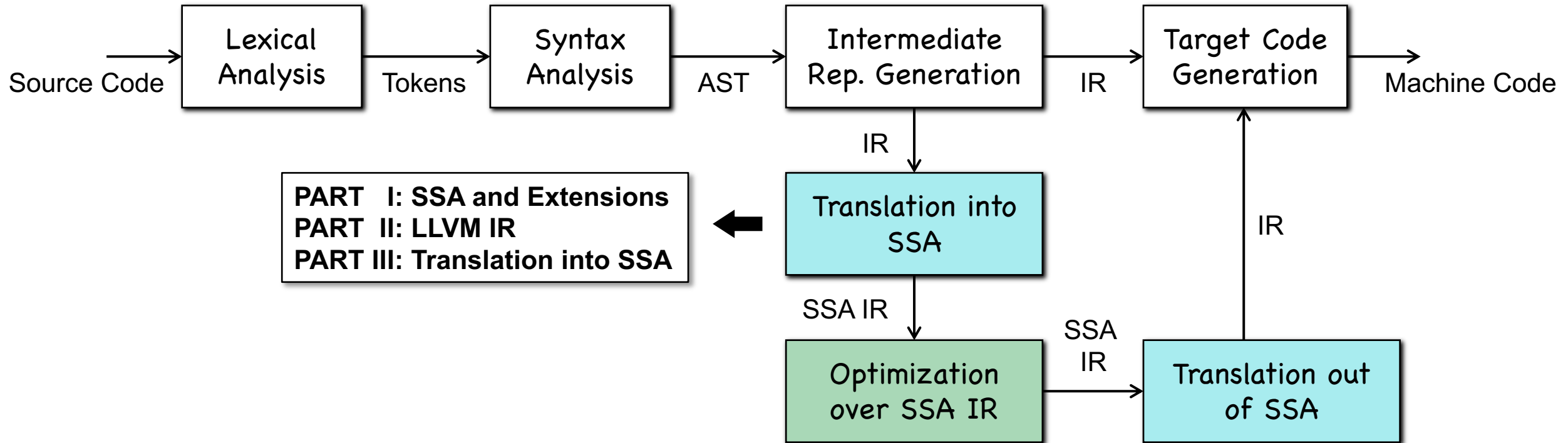


Control Flow Graph



Dominator Tree

Summary



THANKS!